CSE557 · BIG DATA ANALYTICS · CIE-1

The Exam Reader

Units 1, 2 & 3 (CIE-1 part) in one continuous read.
Read it twice before the exam — that's enough to understand and write everything.

1 **Unit 1 · Big Data & Analytics**
Digital data · Big Data & the Vs · BI vs Big Data · Analytics · Data science · SMP/MPP · CAP

2 **Unit 2 · Hadoop**
Why Hadoop · MapReduce · HDFS · File read/write · MR1 · YARN · Failures · Scheduling · Shuffle

3 **Unit 3 · Apache Spark (CIE-1 part)**
Components · Terminology · Architecture · Job execution · Installation · Cluster managers

From your lecture slides + Acharya & Chellappan · Tom White · Abbasi

Exam-eve edition

How to read this (2 passes)

This reader is written as **one story**, not as notes. Every topic follows the same order, so your brain knows what's coming:

1 · Plain words what it really means → **2 · Definition** the exam sentence → **3 · Key points** what to write → **4 · Diagram** what to draw → **5 · Trick / trap** how to remember

📖 PASS 1 — UNDERSTAND (≈ 3 HRS)
Read slowly from start to end. Read the **"In plain words"** box first, then the rest. At each diagram, stop and say out loud what each arrow means. Don't try to memorise yet.

⚡ PASS 2 — LOCK IT IN (≈ 1.5 HRS)
Read only the **definitions**, **bold words**, highlights, tables and 🧠 tricks. For the diagrams marked ✏️ **must-draw**, cover the figure and draw it from memory.

The boxes

🗣 IN PLAIN WORDS
The idea in everyday language — read this first.

🔑 KEY POINT
The one thing to remember if you forget the rest.

🧠 MEMORY TRICK
Mnemonic for a list or an order of steps.

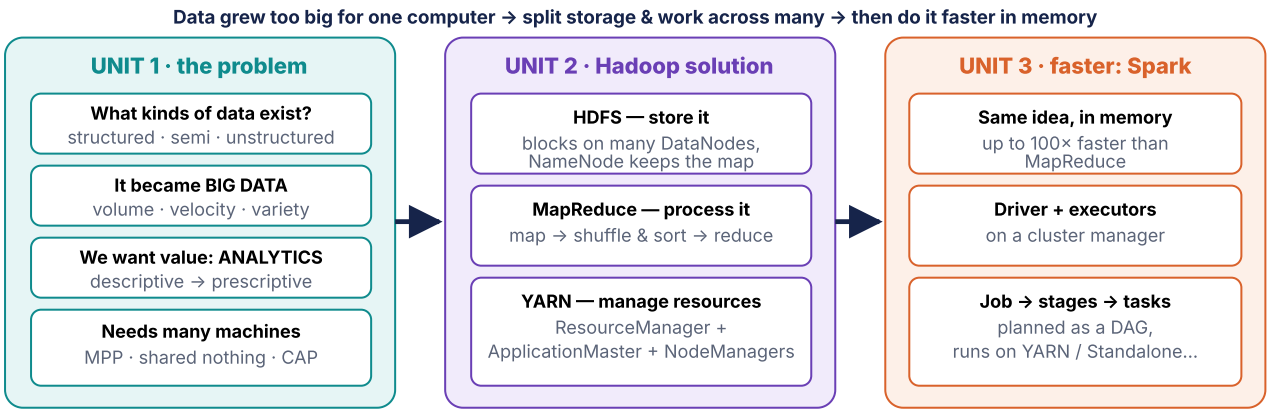
⚠️ TRAP
Things students mix up and lose marks on.

📌 EXAMPLE
A real-life case to quote in answers.

🎯 EXAM
How it's asked and how to answer it.

The whole course in one picture

Before the details, here is how the three units connect. Keep this story in mind — every topic fits somewhere in it.



The story of CIE-1: problem (Unit 1) → Hadoop's answer (Unit 2) → Spark's faster answer (Unit 3)

Contents

Unit 1 · Big Data & Big Data Analytics 4

1.1 Data and its three types 4

1.2 Structured data 5

1.3 Semi-structured & unstructured data 6

1.4 What Big Data is (3 Cs, evolution, definition, Vs) 9

1.5 Challenges, why Big Data, BI vs Big Data 11

1.6 Big Data Analytics & its classification 12

1.7 Data science & the data scientist 15

1.8 Terminologies: in-memory, in-DB, SMP, MPP, shared nothing 16

1.9 CAP theorem 18

Unit 2 · Data Analytics with Hadoop 19

2.1 Why Hadoop? What is Hadoop? 19

2.2 MapReduce with the weather example 20

2.3 Scaling out: splits, locality, data flow 22

2.4 HDFS design, blocks, NameNode & DataNodes 23

2.5 Federation & High Availability 25

2.6 Anatomy of a file read & write 26

2.7 Workflows & unit tests 28

2.8 How a job runs: classic MapReduce & YARN 28

2.9 Failures & job scheduling 30

2.10 Shuffle and sort 32

Unit 3 · Apache Spark Overview (CIE-1 part) 33

3.1 What Spark is & its components 33

3.2 Terminology: SparkContext, job, stage, task 34

3.3 Spark architecture 35

3.4 How a Spark job executes 36

3.5 Installing & running Spark, cluster managers 37

Exam pack 39

UNIT 1

Big Data & Big Data Analytics

The story of this unit

Data comes in three shapes. Most of it is messy (unstructured). There is now so much, arriving so fast, in so many shapes, that we call it **Big Data**. Analysing it — **Big Data Analytics** — gives businesses better decisions. Doing that needs **data scientists** and special **architectures** (many machines, sharing nothing), which run into the **CAP trade-off**.

Textbook: Seema Acharya & Subhashini Chellappan

1.1 Data and its three types

IN PLAIN WORDS

Data = raw facts a computer can store and work on. Organise it and it becomes **information**; understand it and it gives **insights**.

DEFINITION — DATA

The quantities, characters or symbols on which operations are performed by a computer, which may be stored and transmitted as electrical signals and recorded on magnetic, optical or mechanical recording media.

- **Quantitative** (numbers) — **discrete** = counted (20 dogs), **continuous** = measured (18 cm). **Qualitative** = descriptive (red, soft).
- Data sits **inside** the enterprise and **outside** its firewall, in **homogeneous** (same kind) and **heterogeneous** (mixed) sources.
- Computer data = text, images, audio, programs… processed by the CPU, stored in files/folders.

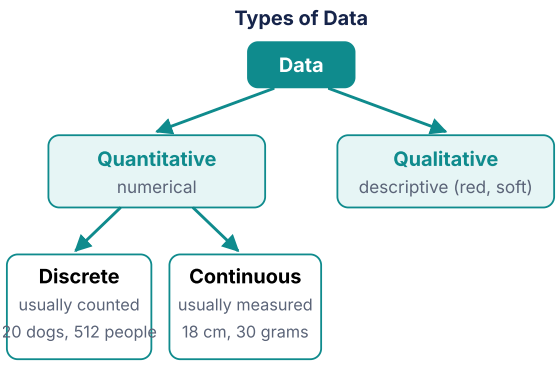


Figure 1 — Types of data

Digital data comes in **three types**. Think of three cupboards: labelled boxes in neat rows (**structured**), items with name-tags but no fixed boxes (**semi-structured**), and a pile (**unstructured**).

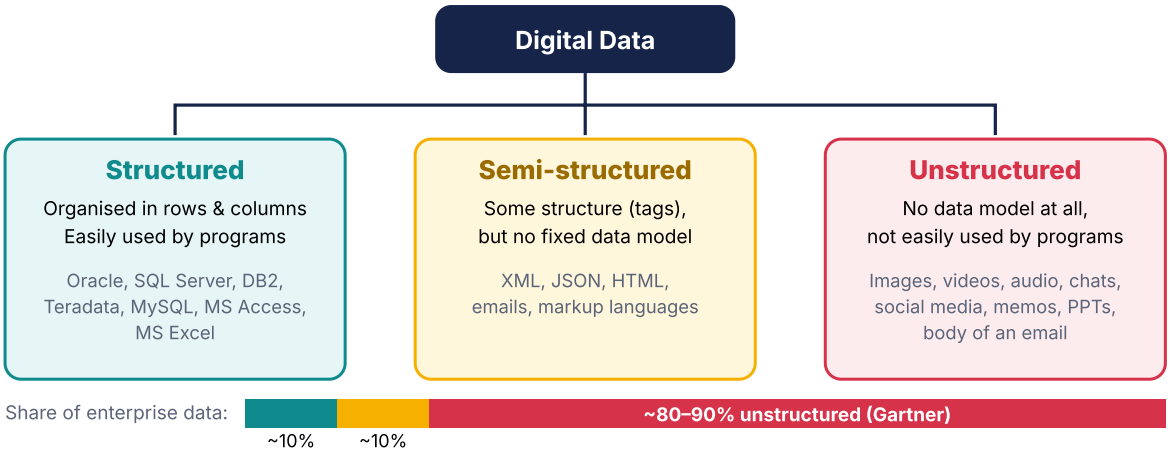


Figure 2 — ✏️ must-draw · Classification of digital data and their share of enterprise data

🔑 KEY POINT

80–90% of an organisation's data is unstructured (Gartner: 80%). That is why ordinary databases are not enough and Big Data tools exist.

1.2 Structured data

DEFINITION

Data in an organised form (rows and columns) that can be easily used by a computer program; relationships exist between entities (like classes and objects). Example: data stored in databases.

- Lives mostly in an **RDBMS** (Oracle, MySQL, SQL Server, DB2, Teradata, PostgreSQL; also MS Access, Excel, OLTP systems).
- A table = a **relation**. Cardinality = number of **rows**; degree = number of **columns**.
- **Designing it:** (1) choose columns and data types; (2) add **constraints** — UNIQUE, NOT NULL, business rules (e.g., value ≥ 50), permissible values (e.g., only CS/IS/MS); (3) **relate tables** through a common column (primary key ↔ foreign key).

📌 EXAMPLE — EMPLOYEE TABLE

EmpNo (**primary key**), EmpName, Designation (NOT NULL), DeptNo, ContactNo (NOT NULL). Rows: E101 Allen, E102 Simon → cardinality 2, degree 5. It links to a Department table through **DeptNo**.



Figure 3 — Tables related through a common column

Why structured data is easy to work with — 5 reasons

Reason	Meaning
Insert / update / delete	DML statements make input, storage, access and processing easy
Security	Encryption and tokenization; only authorised people see sensitive data
Indexing / searching	An index speeds up retrieval (SELECT), at the cost of extra storage and writes
Scalability	Scale the server up — more CPU, RAM, disk
Transaction processing	Supports **ACID**: **A**tomicity (all or nothing) · **C**onsistency (valid state → valid state) · **I**solation (runs as if alone) · **D**urability (changes are permanent)

🧠 **MEMORY TRICK**

ISIST — Insert, Security, Indexing, Scalability, Transactions. Cardinality = **C**ount of rows. ACID with a bank transfer: money leaves and arrives together (A), totals stay right (C), parallel transfers don't mix (I), a power cut can't undo it (D).

1.3 Semi-structured and unstructured data

Semi-structured data

DEFINITION

Data that does not conform to a data model but has some structure; not easily used by a program. Also called a self-describing structure. Examples: emails, XML, HTML, JSON.

5 features

1. Doesn't follow the relational data model.
2. Uses **tags** to separate meaning (semantic elements).
3. Tags also build **hierarchies** of records and fields.
4. **No separation between data and schema.**
5. Entities of the same class can have **different attributes**, in any order.

4 characteristics

1. Inconsistent structure.
2. Self-describing — **label/value pairs**.
3. Schema information blended with data.
4. Attributes may not be known beforehand.

Sources: XML (popularised by **SOAP** web services), JSON (sends data between server and web app; popularised by **REST**; **MongoDB** and **Couchbase/CouchDB** store JSON natively), HTML.

📌 **SELF-DESCRIBING JSON**

`{ _id: 9, BookTitle: "Fundamentals of Business Analytics", AuthorName: "Seema Acharya", Publisher: "Wiley India" }` — each value carries its own label. An **email** is semi-structured: the header (To, From, Subject) has fields, the body is free text.

Unstructured data

DEFINITION

Data that does not conform to a data model or is not in a form easily used by a computer program. 80–90% of organisational data. Examples: memos, chat rooms, PPTs, images, videos, letters, research papers, body of an email.

It is un-organised, has no predefined model, cannot be stored in rows and columns, and is hard to search (e.g., storing users' Aadhaar images, CCTV footage, social media posts).

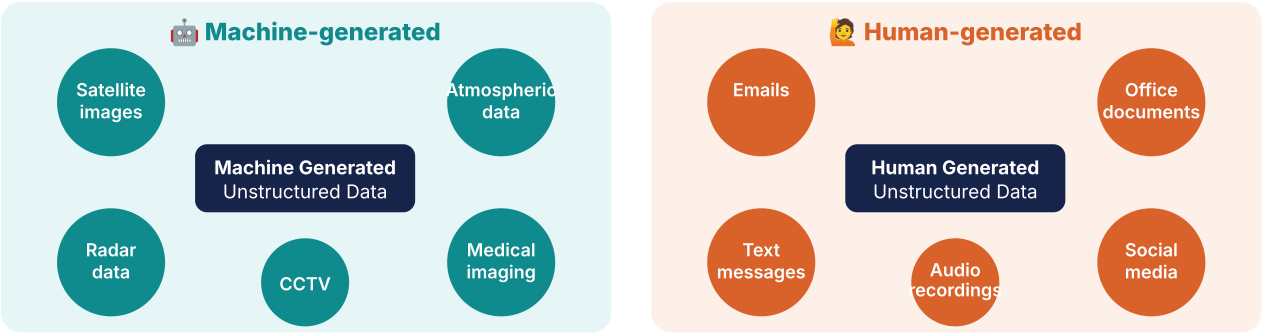


Figure 4 — Unstructured data is machine-generated or human-generated

Why "unstructured" is an imperfect label (3 issues): structure may be **implied** but not formally defined · data may have structure that **doesn't help** the task at hand · data may have structure that is **unexpected** or unannounced.

How we find patterns in unstructured data — 7 techniques



Figure 5 — ✏️ must-draw · Techniques for dealing with unstructured data

Technique	What it does	Example
Data mining	Finds patterns in large data — "knowledge discovery in databases" (KDD). Algorithms: **association rules**, **regression**, **collaborative filtering**	Bread buyers also buy butter; marks from study hours; "people like you liked..."
Text analytics / mining	Patterns from text: categorisation, clustering, sentiment analysis, entity extraction	10,000 reviews → "battery life is the main complaint"
NLP	Computers understanding human language	Google Translate, chatbots, spam filters
Noisy text analytics	Useful info from messy text — typos, slang, abbreviations	"hey!!! u coming tmrw??"
Manual tagging with metadata	A person adds tags (metadata = data about data)	Article tagged Sports, Cricket, India
POS tagging	Grammatical role of each word	Rahul (noun) plays (verb) cricket (noun)
UIMA	Unstructured Information Management Architecture — open-source, by **IBM**; extracts info from text (content analytics)	"Apple" → organisation, "California" → location

🧠 **MEMORY TRICK**

"**D**ark **T**ea **N**ever **N**eeds **M**ore **P**ure **U**pgrades" — Data mining, Text analytics, NLP, Noisy text, Manual tagging, POS tagging, UIMA.

The big comparison

Property	Structured	Semi-structured	Unstructured
Technology	Relational tables	XML / RDF	Character & binary data
Transactions	Matured, with concurrency	Adapted from DBMS, not matured	None
Versioning	Over tuples, rows, tables	Over tuples or graphs	As a whole
Flexibility	Schema-dependent, least	Medium	Most (no schema)
Scalability	Very difficult	Simpler	Most scalable
Robustness	Very robust	New, not widespread	—
Query	Complex joins	Queries over anonymous nodes	Only textual queries

🧠 **READ THE TABLE AS A PATTERN**

Moving **structured → unstructured**: **flexibility and scalability go up**; robustness, transactions and query power go **down**. Rows: "**T**he **T**iny **V**ampire **F**lew **S**wiftly, **R**arely **Q**uerying".

1.4 What Big Data is

Characteristics of any data — the 3 Cs:

- **Composition** — its structure: sources, granularity, types, static or real-time streaming.
- **Condition** — its state: usable as is, or needs cleansing?
- **Context** — where and why it was generated.

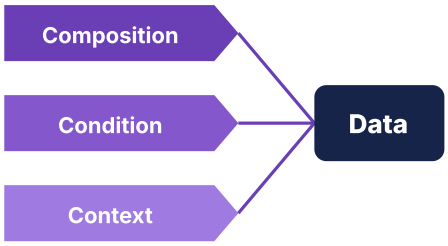


Figure 6 — The 3 Cs of data

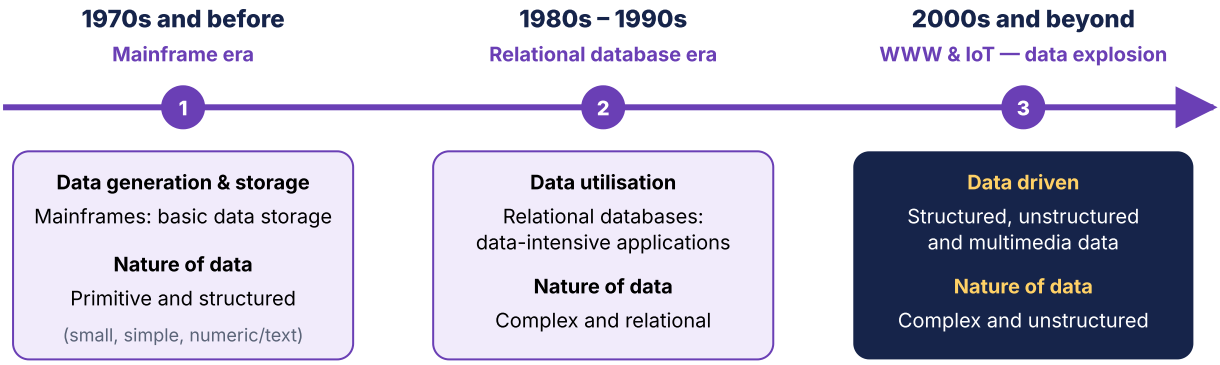


Figure 7 — Evolution of Big Data: mainframes → RDBMS → WWW & IoT

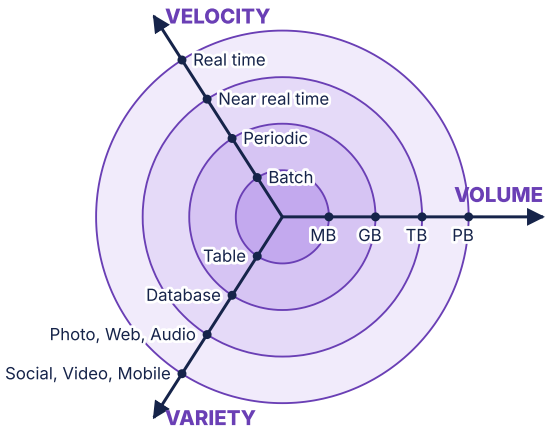
Data went from **primitive and structured** (mainframes, ≤1970s) to **complex and relational** (RDBMS, 1980s–90s) to **complex and unstructured** (web, IoT, social media, 2000s+). The web and IoT caused the explosion we call Big Data.

GARTNER DEFINITION — LEARN WORD-FOR-WORD

"Big Data is high-volume, high-velocity and high-variety information assets that demand cost-effective, innovative forms of information processing for enhanced insight and decision making."

Three parts: **what it is** (3 Vs) → **what it needs** (new processing) → **what it gives** (insight, decisions). Chain: data → information → actionable intelligence → better decisions → enhanced business value. Informal views from the slides: "anything beyond our infrastructure", "today's BIG is tomorrow's NORMAL", "TB, PB, ZB of data", "the 3 Vs".

The 3 Vs



- **Volume — how much.** KB → MB → GB → TB → PB → EB → ZB → YB, each ×1024 (≈ ×1000).
- **Velocity — how fast.** Batch → periodic → near real time → real time (e.g., card-fraud check at swipe).
- **Variety — how many kinds.** Structured + semi-structured + unstructured.

> 🧠 **MEMORY TRICK**
>
> A **river**: how much water (volume), how fast (velocity), how many things floating in it (variety). Units: "**K**ind **M**en **G**ive **T**ea, **P**lease **E**at **Z**esty **Y**ogurt".

Figure 8 — ✏️ must-draw · Big in volume, velocity and variety

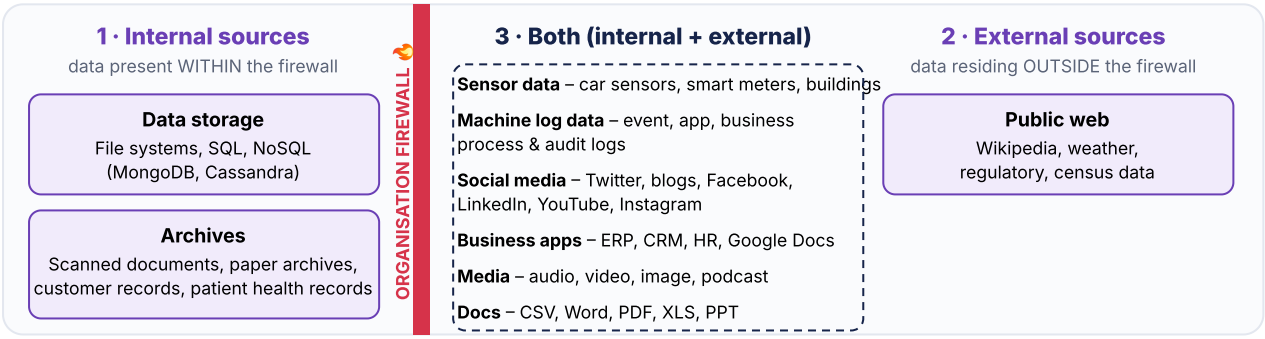


Figure 9 — Where the volume comes from: internal, external and both

Other characteristics (not part of the definition): **Veracity** — can we trust it (noise, bias)? and **Validity** — is it correct for this problem? · **Volatility** — how long it stays useful (stock price today vs birth date) · **Variability** — the flow is irregular, with sudden peaks (festival-sale traffic).

> ⚠️ **TRAP**
>
> **Velocity** (speed of arrival) ≠ **Variability** (irregular flow) ≠ **Volatility** (how long it stays valid).

1.5 Challenges, why Big Data, and BI vs Big Data

Challenges with Big Data (6):

1. **Huge growth** — hard to tell useful data from noise.
2. **Cloud computing & virtualisation** — cheap, flexible infrastructure, but deciding what to host where.
3. **Data retention** — how long to keep data.
4. **Lack of skilled professionals**.
5. **Capture, storage, processing & security** — traditional databases can't cope with data that is large, fast and varied.
6. **Visualisation** — showing it with charts/dashboards; shortage of experts.

> 🧠 **MEMORY TRICK**
> "**G**iant **C**louds **R**arely **L**ack **S**ome **V**iews".

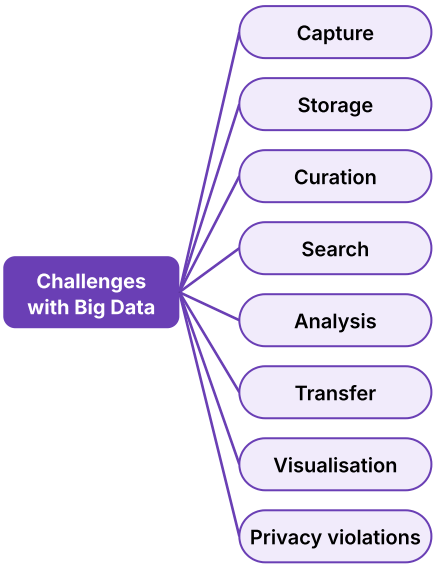


Figure 10 — Challenges with Big Data

Why Big Data? More data → more accurate analysis → more confidence in decisions → greater efficiency: **cost reduction, time reduction, new product development, optimised offerings, smart decisions**. The value lies in how you use the data you own.

Traditional BI vs Big Data

> 🗣 **IN PLAIN WORDS**
> **BI** = the old way: neat company data on one big server, reports about the past. **Big Data** = all kinds of data, from inside and outside, spread over many cheap machines, analysed in real time too.

Point	Traditional BI	Big Data
Storage	Central server	Distributed file system
Scaling	**Vertical** — bigger machine (scale up)	**Horizontal** — more machines (scale out)
Analysis mode	Offline	Real-time and offline
Data	Structured	Variety (all types)
Movement	**Data taken to the processing functions**	**Processing functions taken to the data**
Schema	Schema-on-write (define first)	Schema-on-read (store raw, structure when reading)

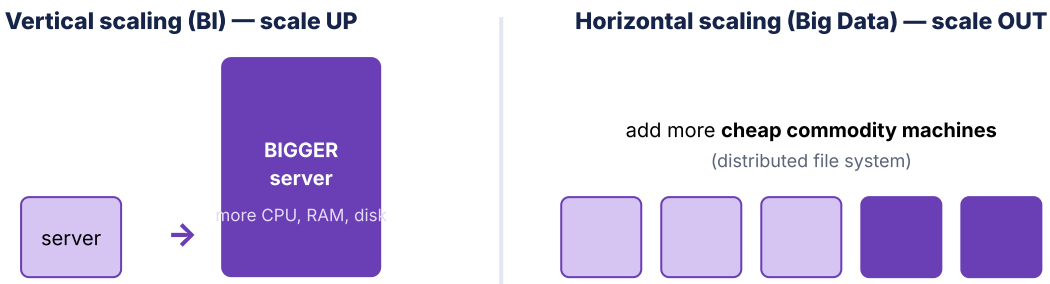


Figure 11 — Scale up (BI) vs scale out (Big Data)

1.6 Big Data Analytics and its classification

📌 **TWO EXAMPLES FROM THE SLIDES**

Amazon: uses your past purchases, browsing history and similar customers' buys to recommend books. **Truck company with 500 trucks:** for an urgent double-pay delivery, it analyses location, load capacity, fuel, traffic, distance and delivery time to pick the right truck. Pattern: **large data → analyse → patterns → better decisions**.

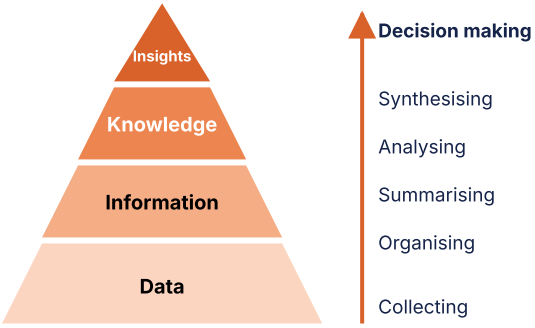


Figure 12 — Raw data becomes actionable insight

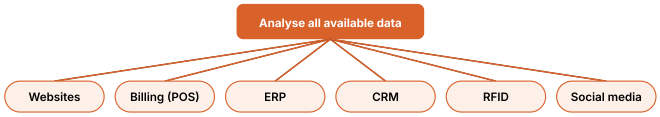


Figure 13 — Where analytics data comes from

Steps: **collect** (data) → **organise** (information) → **summarise & analyse** (knowledge) → **synthesise** (actionable insights) → **decide**.

What Big Data Analytics is — 6 points

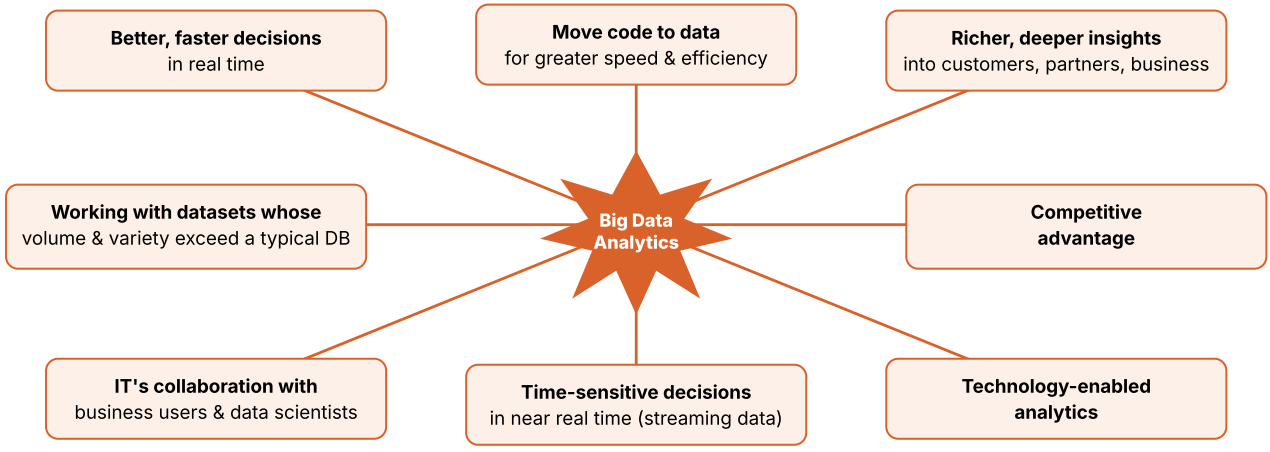


Figure 14 — ✏️ must-draw · What is Big Data Analytics?

1. **Technology-enabled analytics** — tools from IBM, Tableau, SAS, R, WPS.

2. **Deeper, meaningful insights** — who are our customers, what do they buy, what else can we sell?
3. **Competitive edge** — faster, better decisions than competitors.
4. **A tight handshake of three communities** — **IT** (infrastructure), **data scientists** (analysis), **business users** (decisions).
5. **Very large and varied datasets** — volume and variety beyond traditional systems.
6. **Moving code to data** — the program (KB/MB) travels to where the data (TB/PB) is stored.



Figure 15 — Moving code to data

What BDA isn't: only about volume · just technology · a replacement for RDBMS · a replacement for the data warehouse · only for Google/Amazon · a one-size-fits-all RDBMS. **Why the sudden hype:** huge data growth · cheaper storage · analytics tools available.

> 🧠 **MEMORY TRICK**
> "**T**he **D**eep **C**ompetitive **H**andshake **W**ith **M**ovement" — the 6 points in order.

Classification of analytics — two schools of thought

School 1 (BOAM): Basic (reports, slicing and dicing of past data) · **Operationalised** (woven into business processes) · **Advanced** (forecasting with predictive/prescriptive models) · **Monetised** (earns direct revenue).

School 2: Analytics 1.0 → 2.0 → 3.0, each adding a new type of analytics.

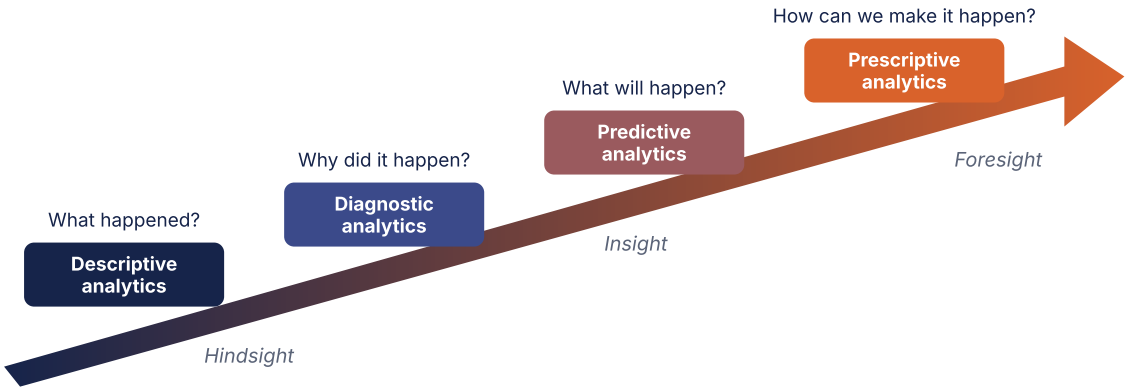


Figure 16 — ✏️ must-draw · Hindsight → insight → foresight

	Analytics 1.0	Analytics 2.0	Analytics 3.0
Era	Mid-1950s – 2009	2005 – 2012	2012 – present
Type	Descriptive	Descriptive + predictive	Descriptive + predictive + prescriptive
Question	What happened? Why?	What will happen? Why?	What will happen, when, why — and what should we do?
Data	Small, structured, internal (data warehouses, ERP, CRM)	Big, mostly unstructured, often external; on Hadoop	Blend of big + legacy data; internal + external
Technology	Relational databases	Database appliances, Hadoop clusters, SQL-to-Hadoop	In-memory, in-database, agile methods, machine learning
Example	"Sales fell last month"	"Sales may rise next month"	"Increase stock next month"

Top challenges facing Big Data (7)

Scale (store & manage growing data) · **Security** · **Schema** (rigid schemas don't fit; need dynamic ones) · **Continuous availability** (24/7) · **Consistency** (users see correct, updated data) · **Partition tolerance** (survive hardware/software failures) · **Data quality** (accuracy, completeness, timeliness, metadata).

🧠 **MEMORY TRICK**

"**S**ix **S**tudents **S**at **C**almly, **C**opying **P**ast **Q**uizzes". Notice C + A + P = the CAP theorem in §1.9.

⚠️ **TRAP**

"Challenges *with* Big Data" (6: growth, cloud, retention…) ≠ "*Top* challenges *facing* Big Data" (7: scale, security, schema…). Read the question wording.

Why Big Data Analytics is important — 4 approaches

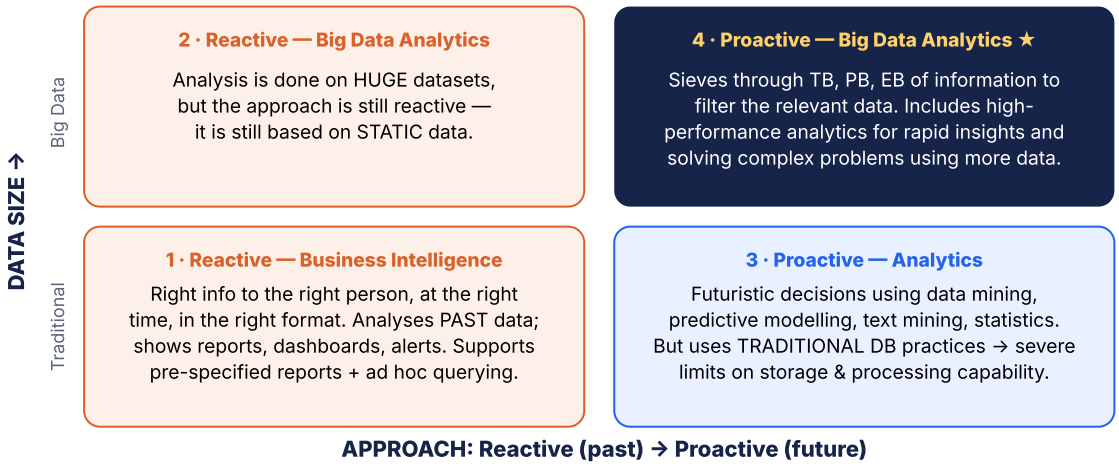


Figure 17 — Reactive vs proactive, traditional vs big data

Reactive = looks at the **past**; proactive = looks at the **future**. Only proactive Big Data Analytics combines future-looking analysis with huge data (TB, PB, EB) — that is why BDA matters: it turns huge data into better decisions, predictions and solutions to complex problems.

1.7 Data science and the data scientist

DEFINITION

Data science is the science of extracting knowledge from data — finding hidden patterns using mathematics, statistics, IT, machine learning, data engineering, probability models and pattern recognition.

Used in weather prediction, oil drilling, seismic activity, financial fraud, social media analytics, retail & manufacturing (churn, market-basket analysis). Process: **collect → prepare → analyse → visualise → communicate → decide**.

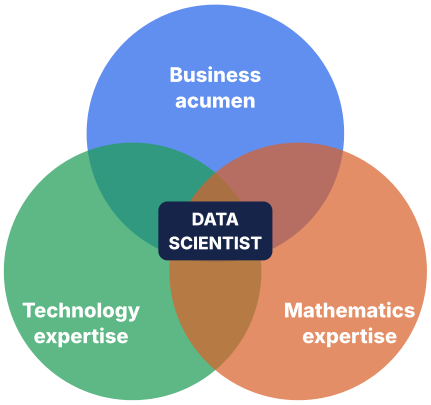


Figure 18 — Three skill sets of a data scientist

- **Business acumen** — domain knowledge, business strategy, problem solving (turn a business question into a data question), communication, presentation, inquisitiveness.
- **Technology** — RDBMS, NoSQL (MongoDB, Cassandra, HBase), programming (Python, Java), data warehousing, data mining, visualisation (Tableau).
- **Mathematics** — maths, statistics, algorithms, machine learning, pattern recognition, NLP.

🧠 **MEMORY TRICK**

BTM — a three-legged stool: remove one leg and it falls.



Figure 19 — The data scientist's 4 responsibilities

1. **Data management** — raw → clean (fix errors, remove duplicates, fill missing) → integrate → ready dataset.
2. **Analytical techniques** — models and algorithms to find relationships, trends and patterns, chosen to fit the question.
3. **Business / domain expertise** — "attendance and marks are related" becomes "low-attendance students need early mentoring".
4. **Communication & presentation** — turn hundreds of numbers into a chart and a clear message.

Data science is a **field**; a **data scientist** is a **person** who applies it. Careers: data analyst, data scientist, ML engineer, data engineer, BI professional, AI/NLP/vision roles.

🧠 **MEMORY TRICK**

"**M**y **A**unt **B**akes **C**akes" — Manage, Analyse, Business context, Communicate.

1.8 Terminologies used in Big Data environments

In-memory analytics & in-database processing

Reading from a hard disk is slow. Pre-computing cubes or aggregate tables helps, but you must guess in advance what you will need. In-memory analytics keeps **all relevant data in RAM**, so nothing has to be fetched from disk. Advantages: faster access, rapid deployment, better insights, minimal IT involvement.



Figure 20 — Disk vs in-memory

In-database processing (in-database analytics): normally OLTP data is cleaned through **ETL** into an **Enterprise Data Warehouse**, then **exported** to a separate analytics program. In-database processing runs the computation **inside the database itself** — no export, saving time.

💡 **THE PAIR**

In-memory brings data **closer to the CPU**; in-database brings computation **to where the data lives**. Both cut data movement.

SMP vs MPP

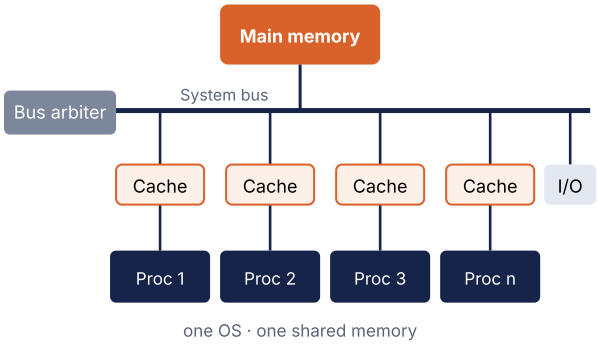


Figure 21 — SMP: one shared memory, one OS

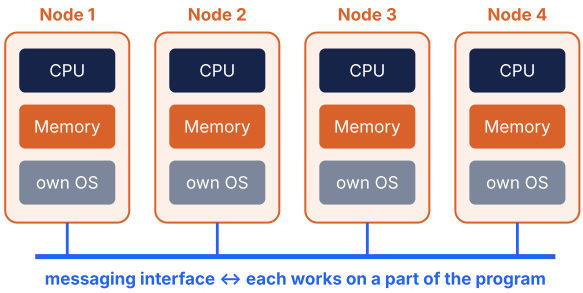


Figure 22 — MPP: each node has its own memory and OS

	SMP — Symmetric Multiprocessor	MPP — Massively Parallel Processing
Memory	One common main memory shared by identical processors	Each processor has its own dedicated memory
OS	Single OS instance	Each has its own OS
Coupling	Tightly coupled; caches + system bus	Looser; communicate by messaging
Programming	Easier	Harder — program split into communicating parts

🧠 **MEMORY TRICK**

SMP = "**S**ame **M**emory, **P**lease"; MPP = "**M**y **P**rivate **P**artition".

Parallel vs distributed systems

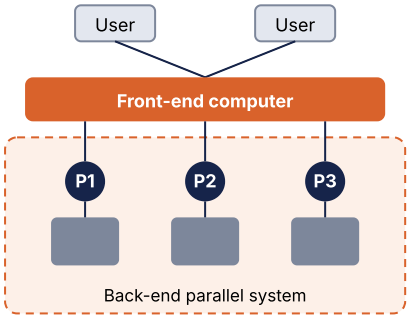


Figure 23 — Parallel system

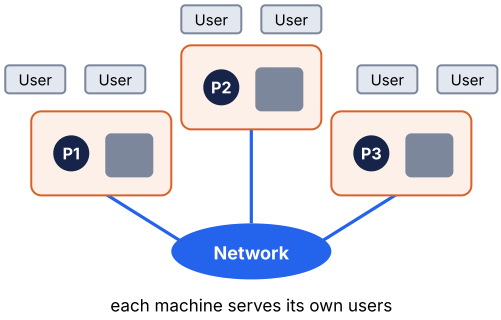


Figure 24 — Distributed system

Parallel: tightly coupled; processors cooperate on the same query; user is unaware of the parallelism; shared memory or message passing. **Distributed:** loosely coupled separate machines on a network; each runs its own applications and serves its own users; data spread across machines, so a query may touch several.

Shared nothing architecture

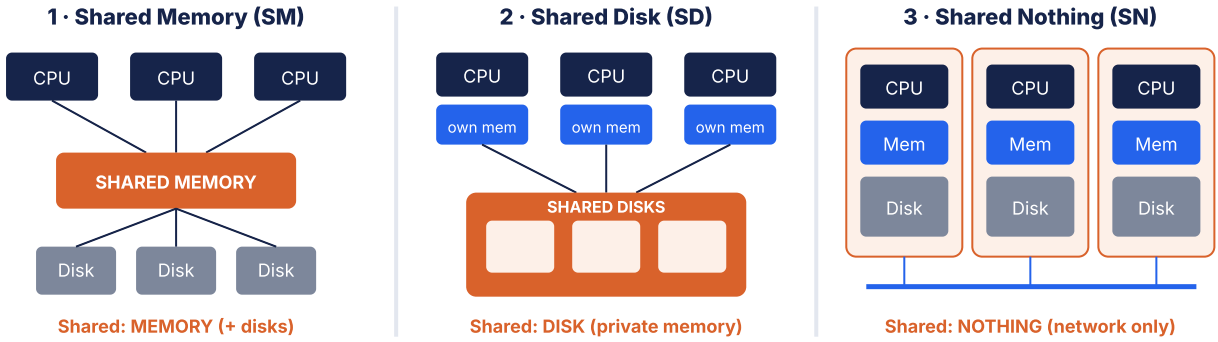


Figure 25 — ✏️ must-draw · Shared memory, shared disk, shared nothing

Three multiprocessor architectures: **shared memory** (common memory), **shared disk** (private memory, common disks), **shared nothing** (neither memory nor disk shared; nodes talk over the network). Advantages of shared nothing:

- **Fault isolation** — a fault in one node stays in that node, visible only through messages (or their absence).
- **Scalability** — with a shared disk, nodes must take turns (synchronisation), which limits how many can be added; with nothing shared there is no central bottleneck. **Hadoop uses shared nothing.**

1.9 CAP theorem

DEFINITION

Also called Brewer's theorem: in a distributed system it is **impossible to guarantee Consistency, Availability and Partition tolerance all at once** — at best you get two of the three.

- **Consistency** — every read gets the **latest write**.
- **Availability** — every request gets a response from a non-failing node in reasonable time.
- **Partition tolerance** — the system keeps working when the **network breaks** between nodes.

Network breaks **will** happen, so P is usually required; the real choice during a partition is **C vs A**.

Pair	Examples
CA	Traditional RDBMS — MySQL, PostgreSQL
CP	HBase, MongoDB, Redis, MemcacheDB, BigTable
AP	Riak, Cassandra, CouchDB, Dynamo

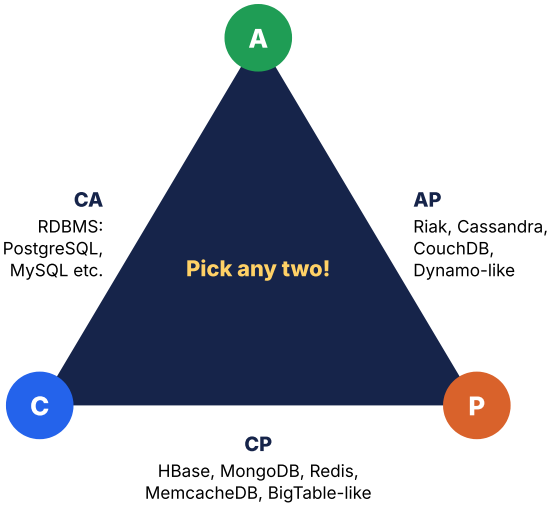


Figure 26 — ✏️ must-draw · CAP triangle with databases

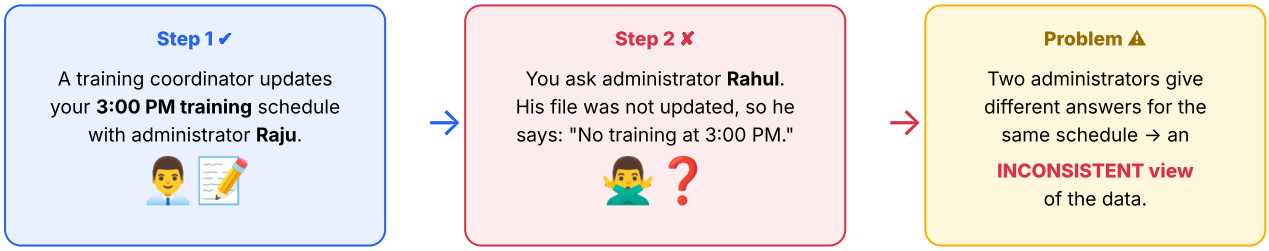


Figure 27 — The training-schedule story: two administrators give different answers = inconsistent

Choose **consistency** when stale data is dangerous (bank balance) — some requests wait. Choose **availability** when the service must keep answering and small differences can be fixed later (social feeds, caches).

🧠 MEMORY TRICK

CP = "**C**orrect **P**lease": "**H**appy **M**onkeys **R**un **M**any **B**ananas". AP = "**A**lways **P**resent": "**R**eal **C**ats **C**hase **D**ogs". CA = classic RDBMS.

UNIT 2

Data Analytics with Hadoop

The story of this unit

One disk is too slow to read Big Data, so we read from **many disks in parallel**. That creates two problems — **machines fail** and **results must be combined**. Hadoop solves the first with **HDFS** (blocks stored 3 times) and the second with **MapReduce** (map → shuffle → reduce). We then see exactly how files are **read and written**, how a job **runs** (classic MapReduce, then **YARN**), how it survives **failures**, how jobs are **scheduled**, and the **shuffle** at its heart.

Textbook: Tom White, Hadoop: The Definitive Guide

2.1 Why Hadoop? What is Hadoop?

Data! The digital universe grew from **0.18 ZB (2006) to 1.8 ZB (2011)** — 1 ZB = 10^{21} bytes = 1 billion TB. NYSE creates ~1 TB a day, Facebook stores ~10 billion photos (≈ 1 PB), the Large Hadron Collider ~15 PB a year. Machines (logs, RFID, sensors, GPS) generate even more. Good news: more data = more value. Bad news: **how do we store and analyse it?**

IN PLAIN WORDS

Disks got **huge** but not much **faster** to read. So the fix is to read from **many disks at the same time**.

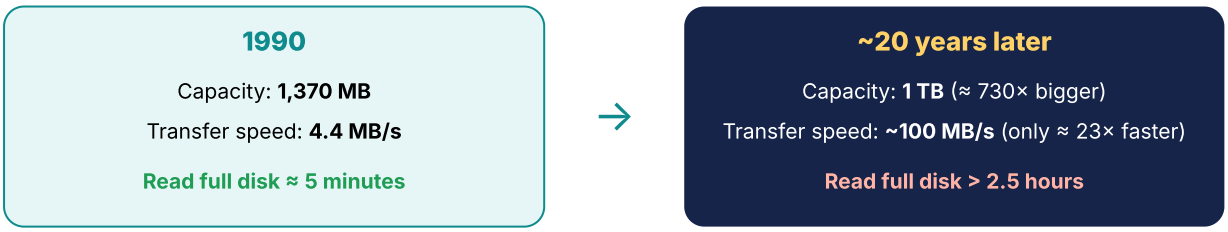


Figure 28 — Capacity grew ~730×, speed only ~23×

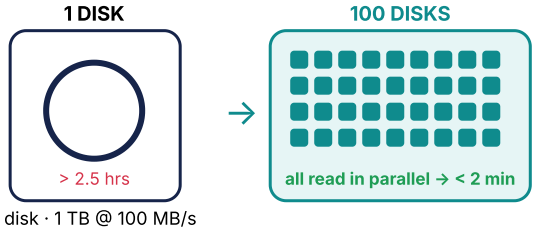


Figure 29 — 1 TB split across 100 disks is read in under 2 minutes

Reading 100 disks in parallel creates **two new problems**:

1 · Hardware fails

More machines = more failures. **Solution: replication** — keep redundant copies. HDFS does this.

2 · Data must be combined

Results from many disks must be merged correctly. **Solution: MapReduce** — map → shuffle → reduce, hiding the coordination.

DEFINITION — HADOOP

An open-source framework that allows us to store and process large data sets in a parallel and distributed manner. In a nutshell: **reliable shared storage + reliable distributed analysis**.

- Written in **Java**; created by Doug Cutting and Mike Cafarella; named after Doug's child's **toy elephant** (not an acronym).
- **Core components: HDFS** — storage layer · **MapReduce** — processing layer · **YARN** — resource management layer · **Common** — shared utilities.
- **Why use it:** ~90% of the Big Data market · scalable · fault tolerant · flexible with any data.
- **History:** Nutch search engine (2002) → Google's GFS paper (2003) → NDFS (2004) → Google's MapReduce paper (2004) → Nutch on MapReduce (2005) → **Hadoop, Feb 2006**. Yahoo! grew it to **24,000 nodes** (2009); Apache top-level project (Jan 2008); sorted **1 TB in 209 s on 900 nodes** (2008).

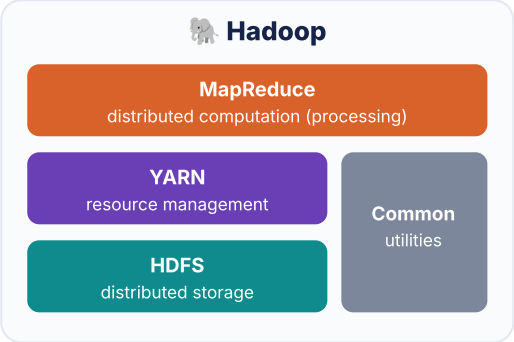


Figure 30 — Hadoop's core components

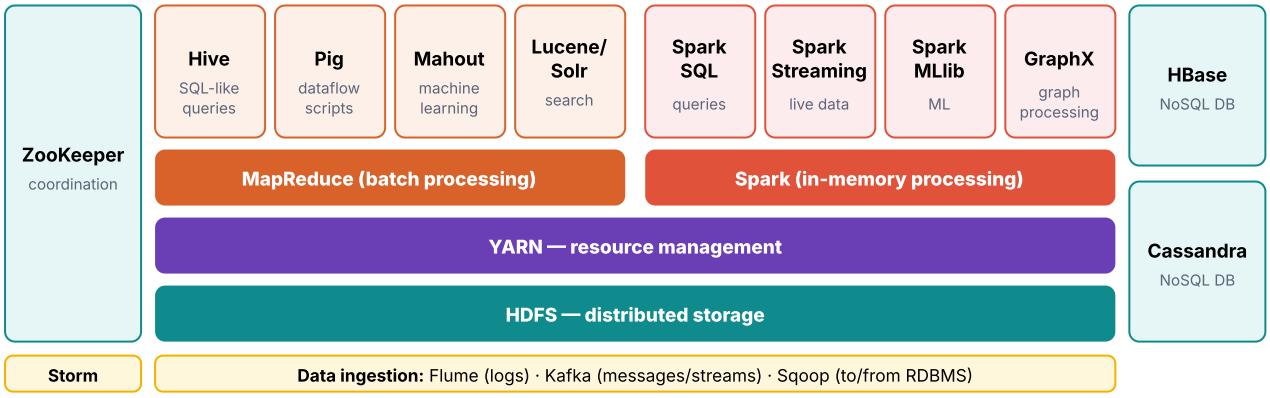


Figure 31 — The Hadoop ecosystem

MEMORY TRICK

"**HMY**": **H**DFS stores, **M**apReduce processes, **Y**ARN manages resources.

2.2 MapReduce with the weather example

IN PLAIN WORDS

Map = each worker reads its piece and writes down only what matters as (key, value) pairs. **Shuffle & sort** = Hadoop collects all values with the same key together. **Reduce** = combine each group into one answer.

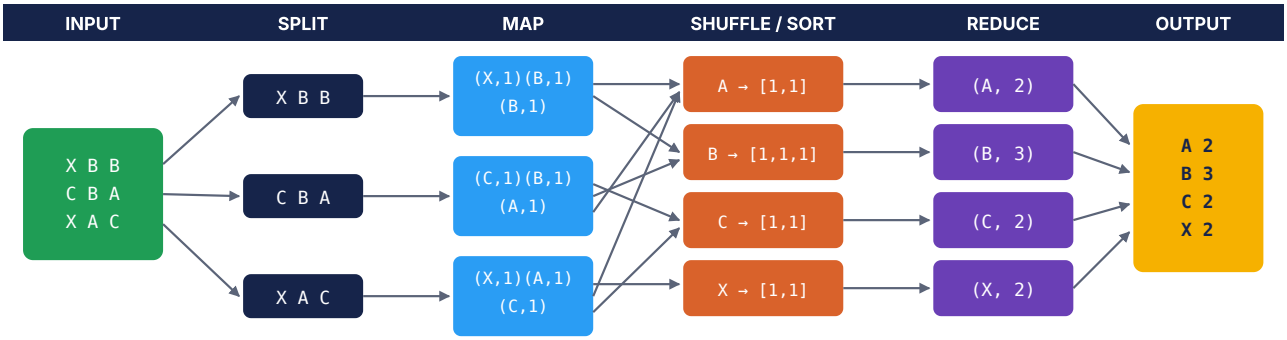


Figure 32 — MapReduce counting letters — input → split → map → shuffle → reduce → output

The dataset: NCDC weather data — semi-structured and record-oriented. Each line is one observation with **fixed-width fields**. Temperature is stored as **°C × 10** (so −0128 = −12.8 °C); **9999** means missing. The original tens of thousands of small station files are **preprocessed into one file per year**, because Hadoop prefers a few large files.

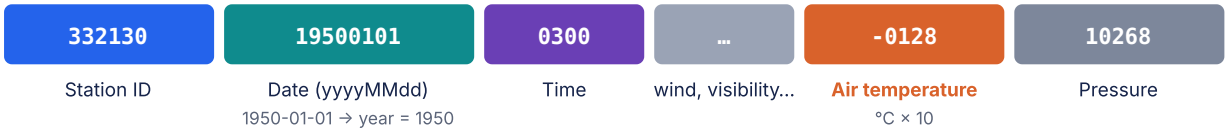


Figure 33 — One weather record

MAP AND REDUCE

MapReduce breaks processing into two phases — map and reduce. Each phase has key–value pairs as input and output. The programmer writes **two functions**: the map function and the reduce function.

Task: find the maximum temperature for each year.

1. **Map** — input is (line offset, line). The mapper extracts **(year, temperature)** and drops bad/missing records: (1950, 0), (1950, 22), (1950, −11), (1949, 111), (1949, 78).
2. **Shuffle & sort** (automatic) — group by key: `1949 → [111, 78]`, `1950 → [0, 22, −11]`.
3. **Reduce** — pick the max of each list: `(1949, 111)`, `(1950, 22)`.

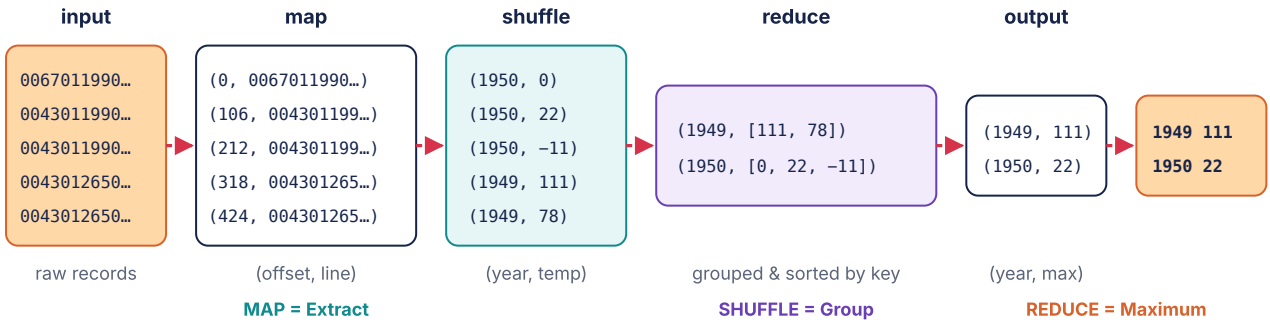


Figure 34 — ✏️ must-draw · MapReduce logical data flow for maximum temperature

The three Java pieces:

- **Mapper** — `Mapper<LongWritable, Text, Text, IntWritable>` = (offset, line) → (year, temp). Reads year at chars 15–19 and temperature at 87–92, skips 9999 and bad quality codes, then `context.write(year, temp)`.
- **Reducer** — `Reducer<Text, IntWritable, Text, IntWritable>`: loops through the values keeping `Math.max`; input types must match the mapper's output types.
- **Driver** — creates a `Job`, sets the JAR, input path, output path (**must not exist**), mapper and reducer classes, output key/value types, then `waitForCompletion(true)`.

```
public void reduce(Text key,
    Iterable<IntWritable> values, Context ctx) {
  int max = Integer.MIN_VALUE;
  for (IntWritable v : values)
    max = Math.max(max, v.get());
  ctx.write(key, new IntWritable(max));
}

% hadoop MaxTemperature input/ncdc/sample.txt
output
% cat output/part-r-00000
1949    111
1950    22
```

Hadoop types: LongWritable ≈ long, Text ≈ String, IntWritable ≈ int. Test first on a 5-record sample.

🧠 **MEMORY TRICK**

Map = Extract · Shuffle = Group · Reduce = Summarise — "**E**very **G**roup **S**ummarised". The programmer writes only map + reduce; Hadoop does distribution, grouping and passing to reducers.

2.3 Scaling out: splits, data locality and data flow

- A MapReduce job = input data + MapReduce program + configuration. Hadoop splits it into **map tasks** and **reduce tasks**, scheduled by **YARN**; a failed task is re-run on another node.
- The input is cut into fixed-size input splits; **one map task per split**. Many small splits = better load balancing (fast machines just do more splits).
- Data locality optimisation: run the map task on the node that **already holds that block**, so no network bandwidth is used.

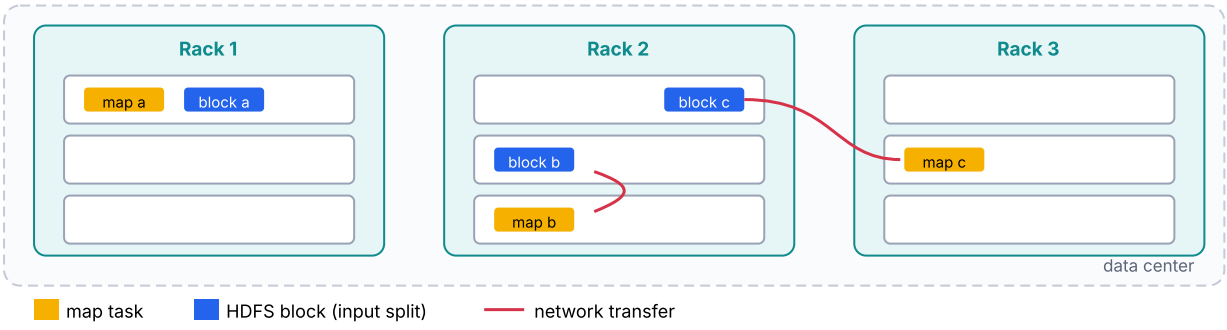


Figure 35 — Data-local (a), rack-local (b) and off-rack (c) map tasks

Order of preference: **data-local** (same node) → **rack-local** (same rack, if the data's nodes are busy) → **off-rack** (other rack, inter-rack transfer). Optimal split size = block size (128 MB), because a block is the largest input guaranteed to be on one node; a split spanning two blocks would need network transfer.

- Map output goes to **local disk** (it is temporary); reducers **copy** it over the network, **merge** it, reduce, and write the result to **HDFS** (replicated).
- With **multiple reducers**, each map partitions its output — one partition per reducer, all values of a key in the same partition, using the default **hash partitioner**. Because every reducer is fed by many mappers, this is called "the shuffle".

- Number of reducers is chosen independently (can even be **zero**). A **combiner** is a mini-reducer run on map output to cut network traffic.

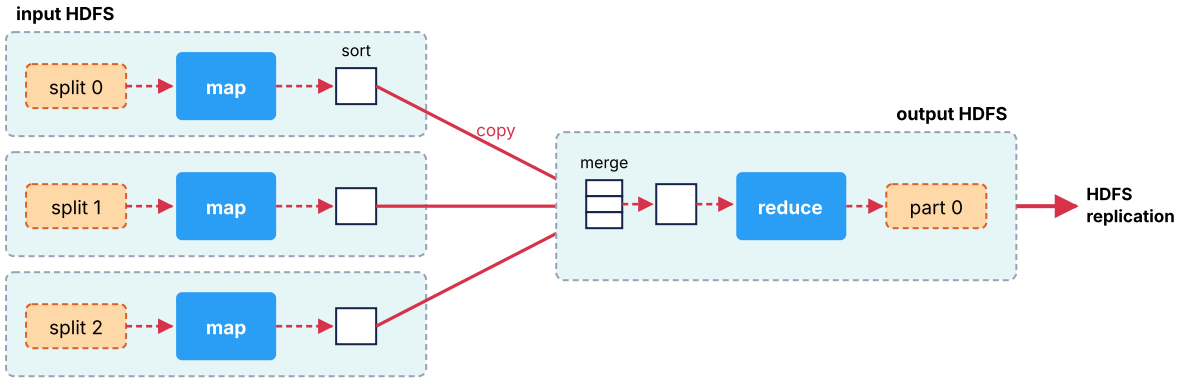


Figure 36 — Data flow with a single reduce task

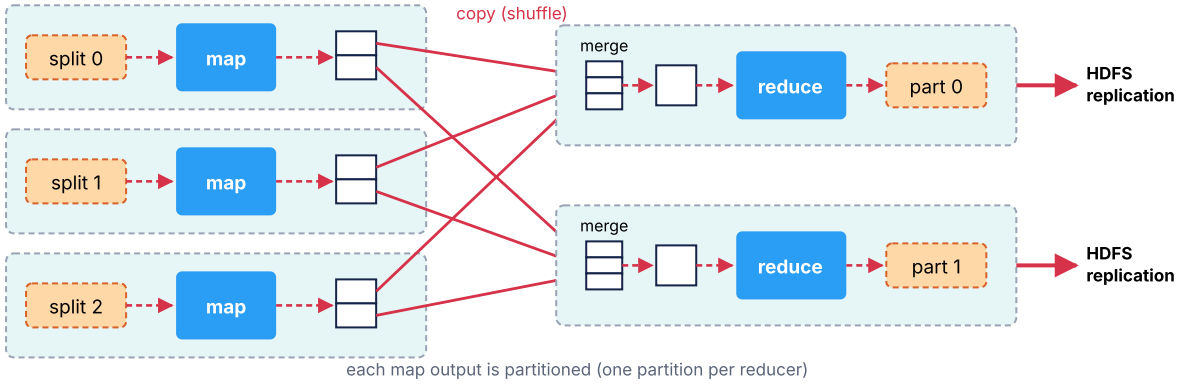


Figure 37 — ✏️ must-draw · Data flow with multiple reduce tasks

🧠 **MEMORY TRICK**

Locality = "**Home → Neighbour → Other town**".

2.4 HDFS: design, blocks, NameNode and DataNodes

When a dataset outgrows one machine, it is partitioned across many. File systems that manage storage across a network of machines are distributed file systems. They are harder than normal file systems because of networking — the biggest challenge is **surviving node failure without losing data**. Hadoop's is **HDFS** (it can also use the local file system or Amazon S3).

DEFINITION — DESIGN OF HDFS (WORD-FOR-WORD)

HDFS is a file system designed for storing very large files with streaming data access patterns, running on clusters of commodity hardware.

HDFS is good at	HDFS is bad at
Very large files — hundreds of MB to TB; clusters hold PB. **Streaming access** — write once, read many times; reading the whole dataset fast matters more than the first record. **Commodity hardware** — cheap machines; keeps working through failures.	**Low-latency access** (tens of ms) — it's built for throughput; use **HBase**. **Lots of small files** — the NameNode keeps metadata in memory: millions OK, billions not. **Multiple writers / random edits** — one writer, writes only at the end of the file.

🧠 **MEMORY TRICK**

Good: "**L S C**" (Large, Streaming, Cheap). Bad: "**L S M**" (Low latency, Small files, Multiple writers).

Blocks

A disk block is 512 bytes, a normal file-system block a few KB, but an HDFS block is 128 MB by default. Files are cut into block-sized chunks stored independently. A file smaller than a block does **not** take a full block (a 1 MB file uses 1 MB).

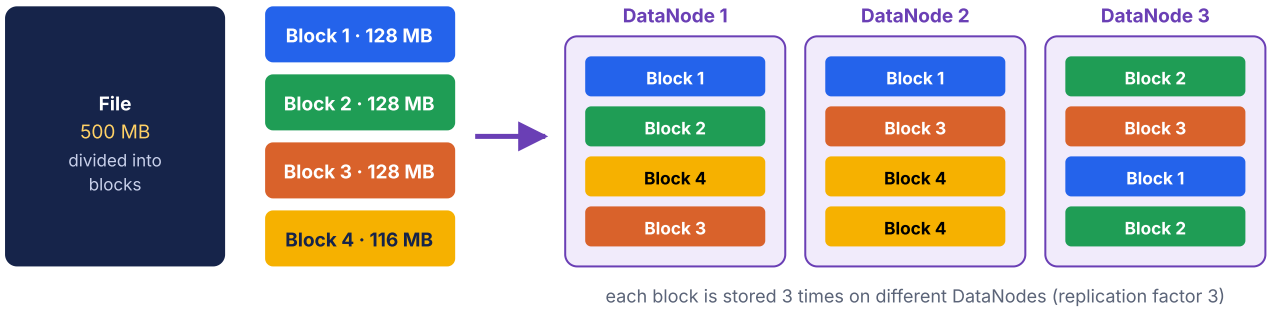


Figure 38 — A 500 MB file → 4 blocks, each stored 3 times on different DataNodes

Benefits of blocks: (1) a file can be **bigger than any single disk**; (2) a fixed-size unit **simplifies storage management**; (3) blocks fit **replication** — each block is copied to **3** separate machines for fault tolerance.

NameNode and DataNodes

🗣 **IN PLAIN WORDS**

A library: the **NameNode** is the **catalogue** (which book is made of which pages, on which shelf); **DataNodes** are the **shelves** holding the pages.

- **Master–worker pattern.** The NameNode (master) manages the **namespace** — the file tree and metadata — stored on disk as the namespace image and the edit log. It learns block locations from DataNode reports.
- DataNodes (workers) are the workhorses: store and retrieve blocks when told, and **report their block lists** to the NameNode periodically.
- **Without the NameNode the file system is unusable** — all files would be lost. So it is protected two ways: (1) write its metadata to **several file systems** at once (local disk + remote **NFS**, synchronous and atomic); (2) run a Secondary NameNode that periodically **merges the namespace image with the edit log** so the log doesn't grow too large; it keeps a copy of the merged image for recovery.

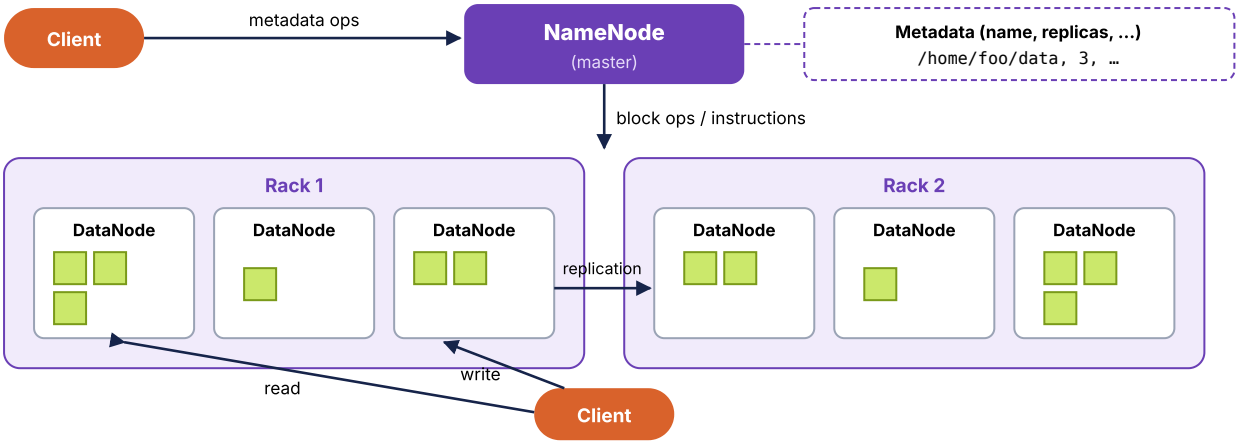


Figure 39 — ✏️ must-draw · HDFS architecture

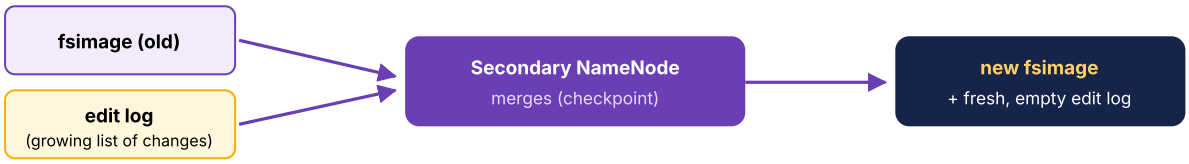


Figure 40 — Checkpointing by the Secondary NameNode

> ⚠️ **TRAP**
> The **Secondary NameNode is NOT a backup NameNode** — it only checkpoints. Automatic takeover is done by the **Standby NameNode** in High Availability (next section).

2.5 HDFS Federation and High Availability

Federation — solves scaling. The NameNode holds every file and block in memory, so memory limits cluster size. HDFS Federation (Hadoop 2.x) **adds NameNodes, each managing part of the namespace** (e.g., one for `/user`, one for `/share`). Each manages a **namespace volume** = metadata + a **block pool**. Volumes are independent: NameNodes don't talk to each other, and one failing doesn't affect the others.

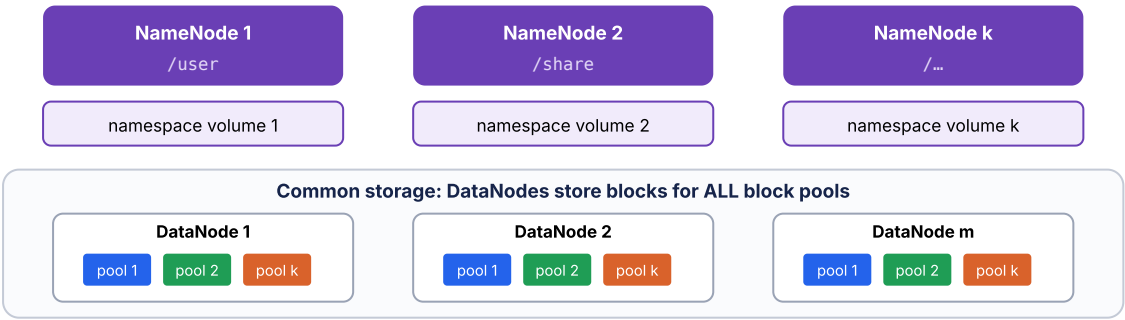


Figure 41 — HDFS Federation

High Availability — solves the single point of failure. Backups and checkpoints prevent data *loss*, but the NameNode is still a single point of failure (SPOF): if it dies, nobody can read, write or list files. Starting a new one means (i) loading the namespace image, (ii) replaying the edit log, (iii) receiving enough block reports to leave **safe mode** — 30+ minutes on big clusters.

Hadoop 2 HA uses an **active–standby pair**; the standby takes over without significant interruption. Needed changes: (1) **shared edit log** on highly available storage; (2) DataNodes send **block reports to both**

NameNodes; (3) clients handle **failover** transparently; (4) the **standby takes the checkpoints** (replacing the Secondary NameNode).

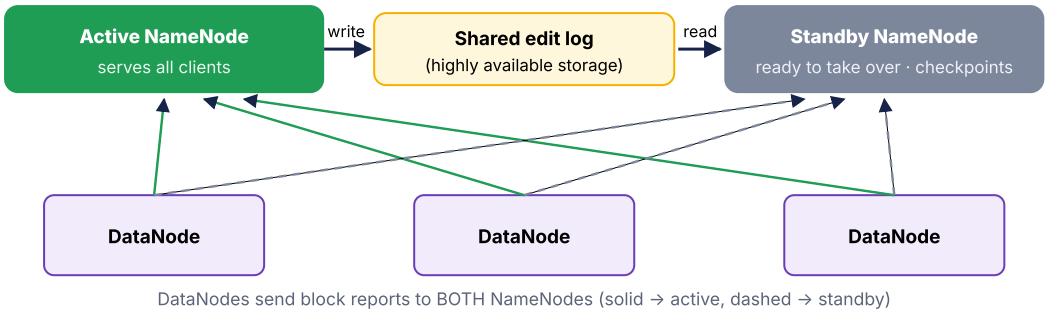


Figure 42 — Active and standby NameNodes

2.6 Anatomy of a file read and a file write

File read — 6 steps

IN PLAIN WORDS

Ask the NameNode **where the blocks are**, then read each block **directly from the nearest DataNode**, one after another. Data never passes through the NameNode.

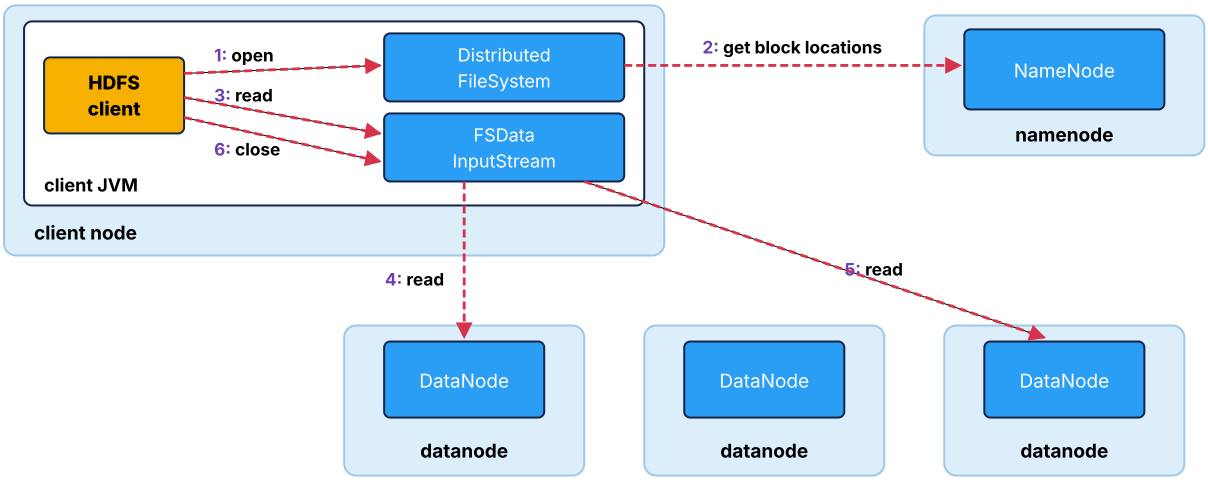


Figure 43 — ✏️ must-draw · A client reading data from HDFS

1. Client calls `open()` on **DistributedFileSystem**.
2. It calls the NameNode by **RPC** for the locations of the first blocks; for each block, DataNodes are returned sorted by closeness. The client gets an **FSDataInputStream** (wrapping a DFSInputStream).
3. Client calls `read()`; the stream connects to the **closest DataNode** for the first block.
4. Data is **streamed** back as the client keeps calling read().
5. At the end of a block, it closes that connection and finds the best DataNode for the **next block** (asking the NameNode for more locations as needed).
6. Client calls `close()`.

Failures: if a DataNode fails, try the **next closest** one and remember the bad node; data is checked with **checksums** — a corrupt block is read from another replica and **reported to the NameNode**. **Why this design is good:** clients go straight to DataNodes, so traffic is spread out and the NameNode only answers quick location lookups.

File write — 7 steps

> **IN PLAIN WORDS**
> Ask the NameNode to **create** the file, then push data in **packets** down a **chain of 3 DataNodes**; each saves and passes it on, and **acknowledgements** come back up.

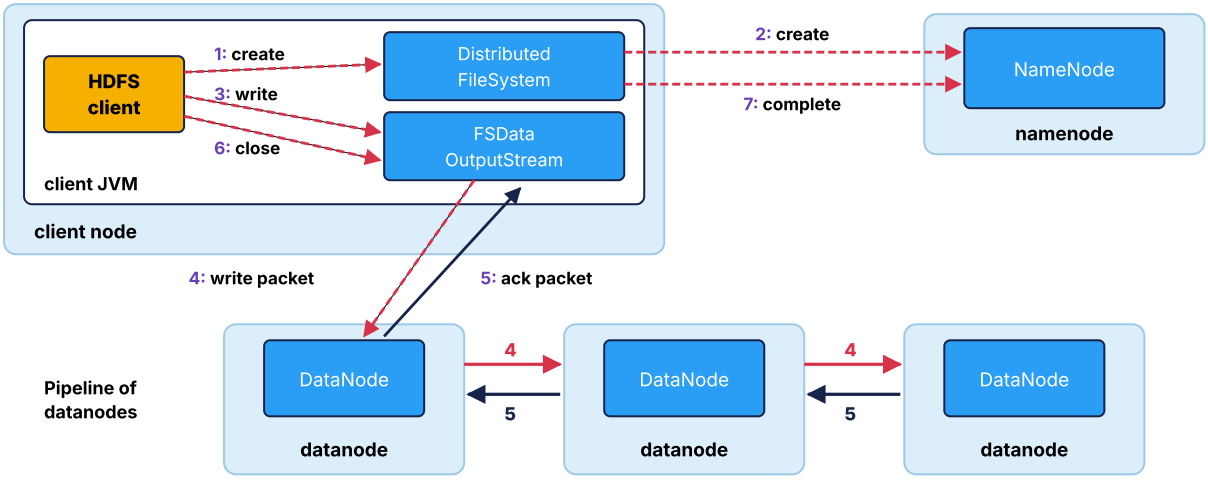


Figure 44 — ✏️ must-draw · A client writing data to HDFS

1. Client calls `create()` on DistributedFileSystem.
2. RPC to the NameNode to create the file (no blocks yet). It checks the file **doesn't exist** and the client has **permission**, else **IOException**. Client gets an **FSDataOutputStream**.
3. As the client writes, data is split into packets placed in the data queue. The **DataStreamer** asks the NameNode for new blocks and a list of DataNodes, which form a pipeline (3 nodes for replication 3).
4. Packets go to DataNode 1, which stores and **forwards** to DataNode 2, then DataNode 3.
5. Packets wait in the ack queue until **all** DataNodes acknowledge them.
6. Client calls `close()` — remaining packets are flushed and acks awaited.
7. The NameNode is told the file is **complete**.

If a DataNode fails during writing: the pipeline is closed; packets in the ack queue go back to the **front of the data queue**; the block on the good nodes gets a **new identity** (so the partial block on the failed node is deleted later); the failed node is removed and a **new pipeline** is built from the two good nodes. The NameNode later adds the missing replica.

	Read	Write
Starts with	`open()`	`create()`
Stream	FSDataInputStream / DFSInputStream	FSDataOutputStream / DFSOutputStream + DataStreamer
Data path	Closest DataNode, block by block	Pipeline of 3 DataNodes, packet by packet
Queues	—	Data queue + ack queue
Steps	6	7

🧠 **MEMORY TRICK**

Read: "**O**nly **G**ood **R**eaders **R**eally **R**ead **C**arefully" (Open, Get locations, Read, stReam, Read next, Close).
Write: "**C**ute **C**ats **W**rite **P**oems **A**nd **C**lose **C**arefully" (Create, Create, Write, Packet, Ack, Close, Complete).

2.7 MapReduce workflows and unit tests

When a problem gets harder, use more MapReduce jobs, not more complex ones (or a higher-level tool: Pig, Hive, Cascading, Crunch, Spark). Example: **mean maximum temperature per station per day** needs two jobs — Job 1 finds the max per (station, date); Job 2 drops the year and averages per (station, day-month): (25 + 27 + 29) ÷ 3 = 27 °C.

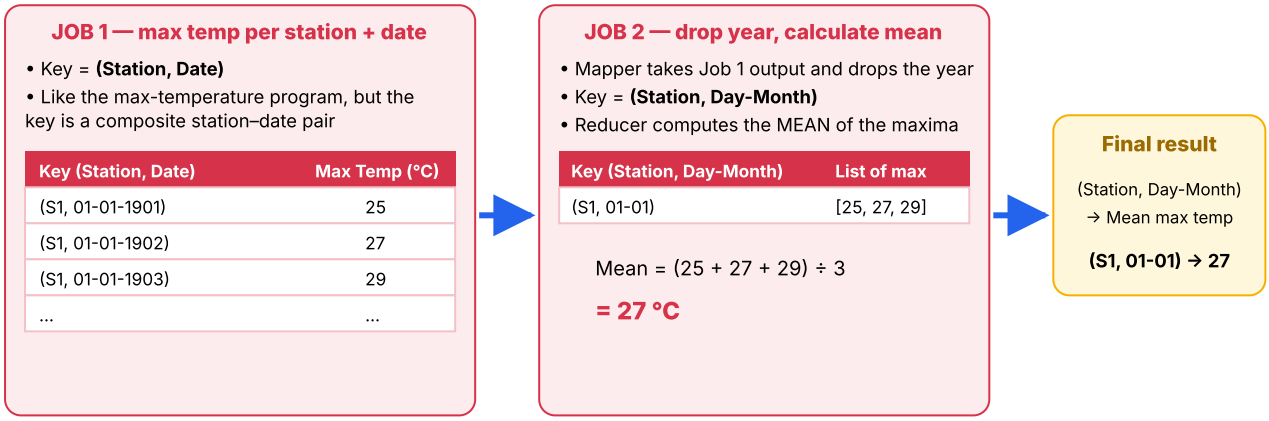


Figure 45 — Breaking a complex problem into two MapReduce jobs

Jobs in a **linear chain** are run one after another with `JobClient.runJob()` (a failure throws IOException and stops the chain). Jobs forming a DAG (directed acyclic graph) are managed by JobControl: it stores jobs, records dependencies, runs them in order, monitors progress, reports status, and skips dependents if a job fails.

MRUnit is a testing library (used with **JUnit**) that feeds known input to a mapper or reducer and checks the output; a mismatch fails the test. **MapDriver** tests the mapper (weather record → (1950, −11)); **ReduceDriver** tests the reducer (1950, [10, 5] → (1950, 10)); the **local job runner** tests the whole job on a small file.

2.8 How a job runs: classic MapReduce and YARN

A job is started by one call — `submit()` or `waitForCompletion()`. The property mapreduce.framework.name chooses how it runs: **local** (whole job in one JVM, for testing), **classic** (MapReduce 1: JobTracker + TaskTrackers), or **yarn** (MapReduce 2).

Classic MapReduce (MR1)

Four entities: the **client** (submits) · the JobTracker (coordinates the job) · TaskTrackers (run the tasks) · the **distributed file system** (shares job files).

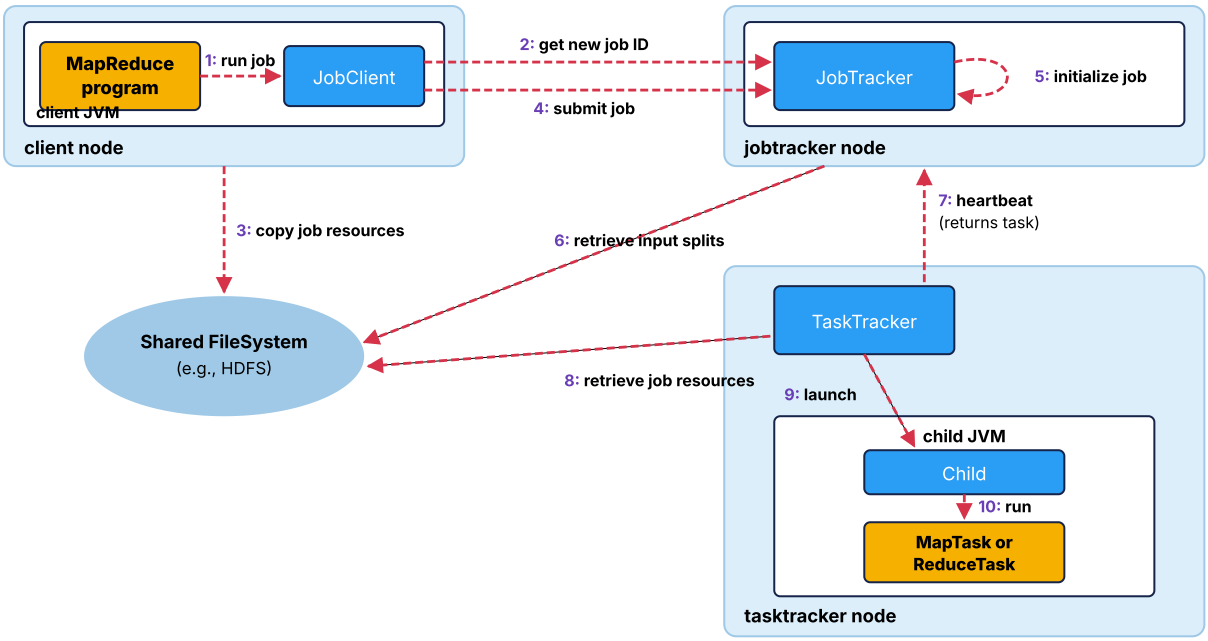


Figure 46 — ✏️ must-draw · Job run in classic MapReduce (steps 1–10)

1. **Submission (1–4):** JobSubmitter gets a **new job ID** from the JobTracker; checks the **output directory** (must not exist) and computes **input splits**; copies the **job JAR** (high replication), config and splits to the shared FS; calls `submitJob()`.
2. **Initialisation (5–6):** job goes into an internal queue; the scheduler creates the job object, retrieves the splits and makes **one map task per split** (reduce count from `mapred.reduce.tasks`).
3. **Assignment (7):** TaskTrackers send heartbeats — "I'm alive, I'm free" — and the JobTracker replies with a task.
4. **Execution (8–10):** the TaskTracker copies the JAR locally (plus distributed-cache files), and a **TaskRunner launches a new JVM** for each task, so bugs in user code can't crash the TaskTracker.

YARN (MapReduce 2)

🗣 IN PLAIN WORDS

In MR1 the JobTracker did two jobs — managing cluster resources **and** babysitting every task — and choked at around **4,000 nodes**. YARN splits it: one **ResourceManager** for the cluster, plus an **ApplicationMaster** for each job.

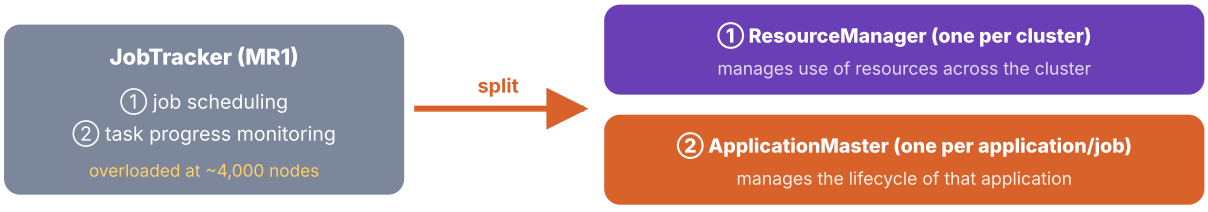


Figure 47 — YARN splits the JobTracker into two daemons

YARN = Yet Another Resource Negotiator (designed at Yahoo!, 2010). **Five entities:** client · ResourceManager (allocates cluster resources) · NodeManagers (launch and monitor containers) · MRAppMaster (coordinates the job's tasks; runs in a container) · distributed file system.

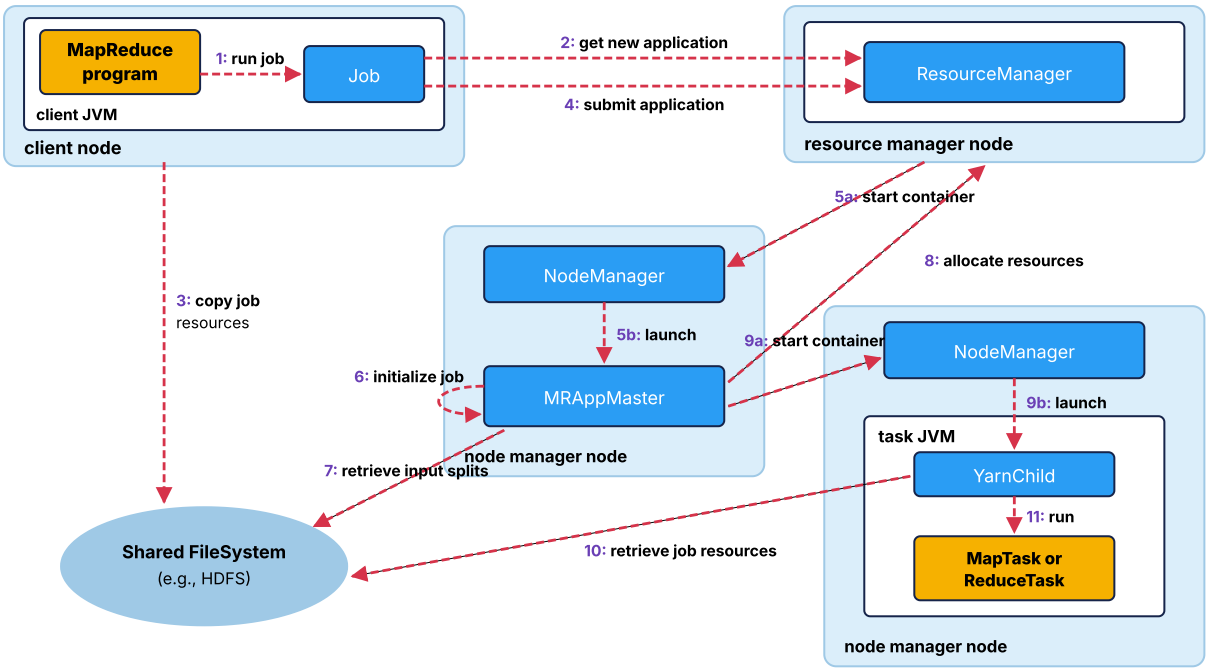


Figure 48 — ✏️ must-draw · Job run on YARN (steps 1–11)

1. **Submission (1–4):** get a new **application ID** from the ResourceManager; check output; compute splits; copy resources; `submitApplication()`.
2. **Initialisation (5a, 5b, 6, 7):** the scheduler allocates a container and the RM launches the **MRAppMaster** in it via a NodeManager; the AM creates bookkeeping objects and retrieves the splits.
3. **Assignment (8):** the AM requests containers from the RM — **map tasks first** (higher priority); reduce tasks only after **5%** of maps finish. (Tiny jobs may run as an **uber task** inside the AM.)
4. **Execution (9a, 9b, 10, 11):** the AM asks a NodeManager to start the container; YarnChild localises the JAR/config, then runs the map or reduce task. Tasks report progress to the AM.

	Classic MapReduce (MR1)	YARN (MR2)
Master	JobTracker — scheduling **and** monitoring	ResourceManager (resources) + per-job ApplicationMaster (monitoring)
Worker	TaskTracker (fixed slots)	NodeManager (flexible containers)
Task runs in	Child JVM from TaskRunner	YarnChild in a container
Entities / steps	4 / 10	5 / 11
Scale	Bottleneck ~4,000 nodes	Much larger
SPOF	JobTracker	Designed to remove SPOFs

🧠 **MEMORY TRICK**

Both runs have 4 phases: "**S**tudents **I**n **A**n **E**xam" — Submission, Initialisation, Assignment, Execution. MR1 entities "**C**lient **J**ust **T**alks to **D**FS"; YARN "**C**an **R**ahul **N**ot **A**ttend **D**aily?"

2.9 Failures and job scheduling

Hadoop notices failures mainly through **missing heartbeats**, then re-runs the work elsewhere.

Failure	Classic MR1	YARN
Task	User code throws an exception or the child JVM exits suddenly → TaskTracker marks the attempt failed (frees the slot). Streaming: non-zero exit code.	Reported to the ApplicationMaster; a hanging task is caught by a missing ping; a task fails after **4 attempts**.
Worker node	TaskTracker stops heartbeats → removed from the pool. **Blacklisted** if more than 4 tasks of a job fail there and its faults are well above average.	NodeManager stops heartbeats → removed after **10 minutes**; the AM blacklists it if more than 3 tasks fail there.
Job coordinator	—	ApplicationMaster fails → RM starts a new one in a new container.
Master	**JobTracker failure = most serious; it's a SPOF → the job fails.**	ResourceManager recovers from **checkpointed state** saved to persistent storage.

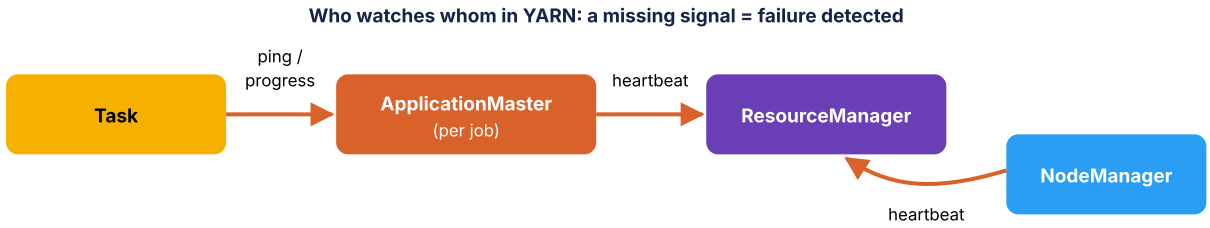


Figure 49 — Who watches whom in YARN

🧠 **MEMORY TRICK**

MR1: "**T**wo **T**ired **J**ugglers" (Task, TaskTracker, JobTracker). YARN: "**T**he **A**nts **N**ever **R**est" (Task, AM, NodeManager, RM).

Job scheduling

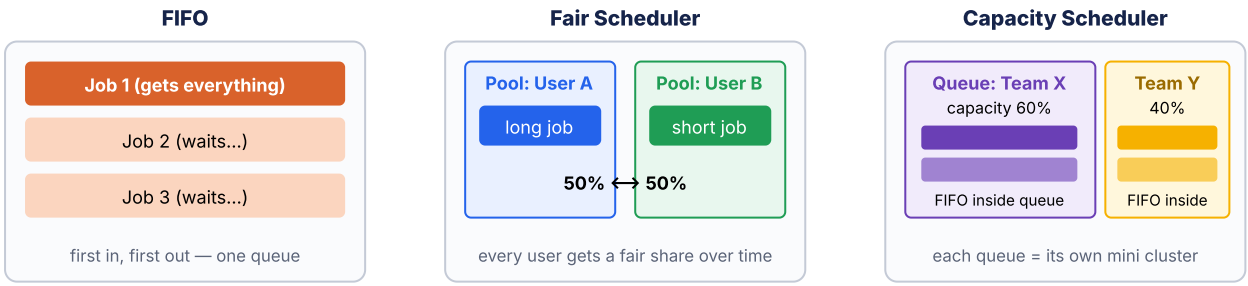


Figure 50 — FIFO vs Fair vs Capacity scheduling

- **FIFO** — default in MR1; one queue, first come first served.
- Fair Scheduler — gives **every user a fair share over time**. A lone job gets the whole cluster; as jobs arrive, slots are shared so short jobs finish even while long jobs run. Jobs go in **pools** (one per user by default). Supports **preemption**: if a pool is starved, tasks in over-capacity pools are killed to free slots.
- Capacity Scheduler — the cluster is split into **queues** (can be hierarchical), each with a **fixed capacity**; inside each queue jobs run **FIFO (with priorities)** — like a separate mini-cluster per team.
- Preemption = forcefully taking resources from one job and giving them to a higher-priority one.

🧠 **MEMORY TRICK**

Fair → **P**ools ("Friends Play fair"); **C**apacity → **Q**ueues ("Companies Queue up").

2.10 Shuffle and sort

DEFINITION

MapReduce guarantees that the input to every reducer is sorted by key. The process of performing this sort and **transferring map outputs to reducers** is the shuffle — "the heart of MapReduce, where the magic happens".

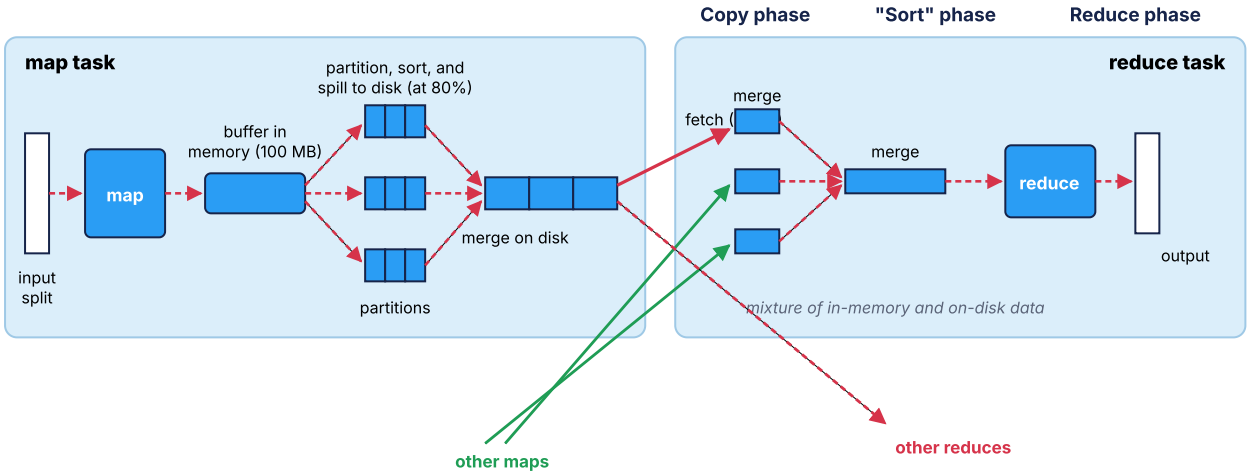


Figure 51 — ✏️ must-draw · Shuffle and sort

Map side

1. Output goes to a circular memory buffer of 100 MB.
2. At 80% full, a background thread **spills** to disk (round-robin into mapred.local.dir); the map blocks only if the buffer fills completely.
3. Before writing, data is **partitioned** by destination reducer.
4. Each partition is **sorted by key in memory**; the **combiner** (if any) runs on it.
5. Several spill files are **merged** into one partitioned, sorted output file (optionally compressed).
6. Partitions are served to reducers over **HTTP**.

Reduce side

1. **Copy phase** — starts copying each map's output **as soon as that map finishes**, using **5** copier threads.
2. Small outputs go to memory, large ones to disk; buffers are **merged and spilled**, and background threads merge disk files.
3. **Sort phase** (really a **merge** phase — sorting was done map-side) — merges outputs in rounds, keeping order.
4. **Reduce phase** — the last merge feeds the reduce function directly; it runs **once per key**; output goes to **HDFS**.

🧠 **MEMORY TRICK**

Map side: "**B**ig **S**pills **P**lease **S**weep, **C**lean, **M**op" — Buffer, Spill, Partition, Sort, Combine, Merge. Reduce side: **Copy → Merge → Reduce**.

UNIT 3 · CIE-1 PART

Apache Spark Overview

The story of this unit

MapReduce writes to disk after every step, which is slow. **Spark** does the same kind of distributed work but keeps data **in memory**. A Spark program is a **driver** that plans the work and **executors** on worker nodes that do it, with a **cluster manager** handing out machines. Your code becomes a **job**, split into **stages** and **tasks**. Finally: how to **install** Spark, write a first program and choose a **cluster manager**.

Slides: "Spark Architecture" · Textbook: Abbasi, Learning Apache Spark 2

Stops before RDDs

3.1 What Spark is and its components

DEFINITION

Apache Spark is an open-source cluster computing framework for data analytics, designed for quick computation. It supports in-memory cluster computing and parallel processing, and is faster than Hadoop.

- A tool for processing large datasets quickly, on **one computer or many**.
- APIs for **Scala, Java, Python** (and SQL, R). Your lab uses **PySpark on one Ubuntu machine**.
- Up to **100× faster than MapReduce in memory** (~10× on disk). Started at UC Berkeley's AMPLab (2009, Matei Zaharia); Apache top-level project 2014.

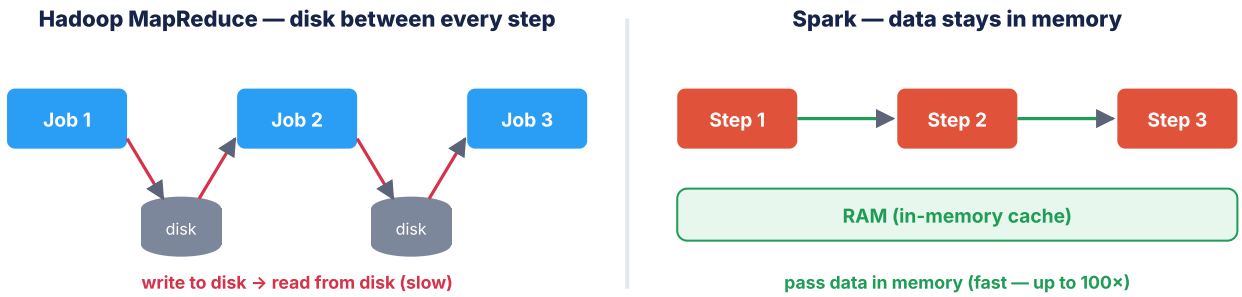


Figure 52 — Why Spark is faster: no disk trip between steps

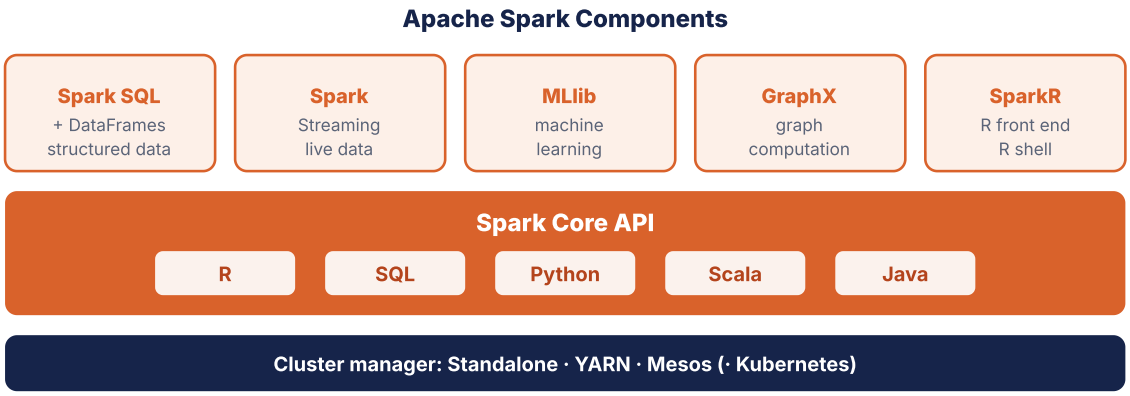


Figure 53 — ✏️ must-draw · The Spark stack

Component	What it does
Spark Core	The **foundation** — execution platform for every Spark application
Spark SQL	SQL and HiveQL queries on structured and semi-structured data (DataFrames)
Spark Streaming	Live data, processed as small **micro-batches**
MLlib	Machine learning; uses in-memory processing for iterative algorithms
GraphX	Graph computation at scale
SparkR	Front end for R users, from the R shell

🧠 **MEMORY TRICK**

"**C**ool **S**tudents **S**tream **M**ovies, **G**raphs & **R**eels" — Core, SQL, Streaming, MLlib, GraphX, SparkR.

3.2 Terminology: SparkContext, job, stage, task

SPARKCONTEXT

The entry point of a Spark application. It connects to the Spark execution environment and is used to create RDDs, accumulators and broadcast variables, access Spark services and run jobs.

Responsibilities of SparkContext

1. Check the current **status** of the application
2. **Cancel** a job or stage
3. **Run** a job synchronously or asynchronously
4. **Access** or unpersist a cached RDD
5. **Dynamic allocation** of resources

From Spark 2.0, **SparkSession** wraps SparkContext as the single entry point.

Spark Shell — a Scala command line with auto-completion, for trying Spark before writing an application (`pyspark` for Python).

Spark application — a self-contained computation running user code; its processes can stay alive even when no job is running.

🧠 **MEMORY TRICK**

"**S**park **C**an **R**un **A**ny **D**ata" — Status, Cancel, Run, Access, Dynamic.

- Task — the **smallest unit of work**, sent to **one executor**; one task per data **partition**.
- Job — a parallel computation of many tasks, created when an **action** (e.g., `count()`) runs.

- Stage — a set of tasks that depend on each other; a job is split into stages where some steps must **wait** for others (a shuffle).

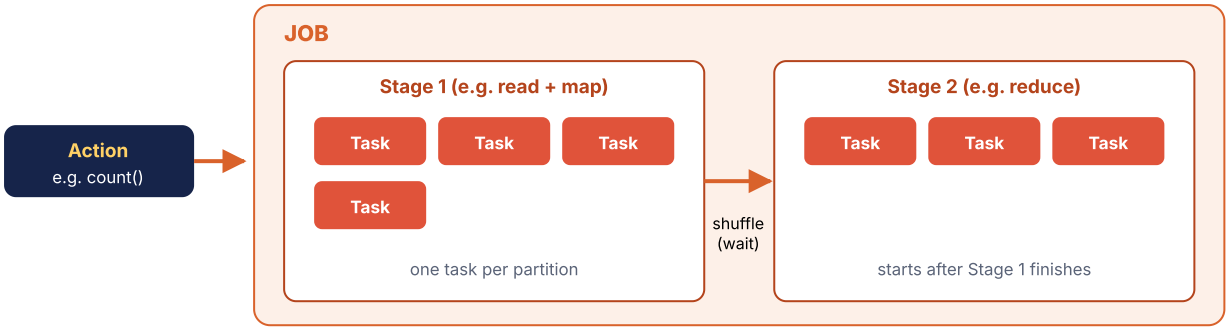


Figure 54 — One action → one job → stages → tasks

> ⚠️ **TRAP**
> A Hadoop job = one map + one reduce. A Spark job can have **many stages**, so it can do the work of several chained MapReduce jobs. Order of size: **Application ⊃ Job ⊃ Stage ⊃ Task** ("All Jobs Split into Tasks").

3.3 Spark architecture

> 🗣 **IN PLAIN WORDS**
> A restaurant: the **driver** is the head chef who reads the order and plans; **executors** are the cooks; the **cluster manager** is the manager who gives each chef cooks and stoves; a **task** is one dish.

Like Hadoop, Spark uses a master–slave architecture: one master and many slaves. The driver is the central coordinator. The driver and each executor run in their **own Java process**. A Spark application = **a driver and its executors**. Spark has its own **standalone cluster manager** and also works with **YARN** and **Mesos**.

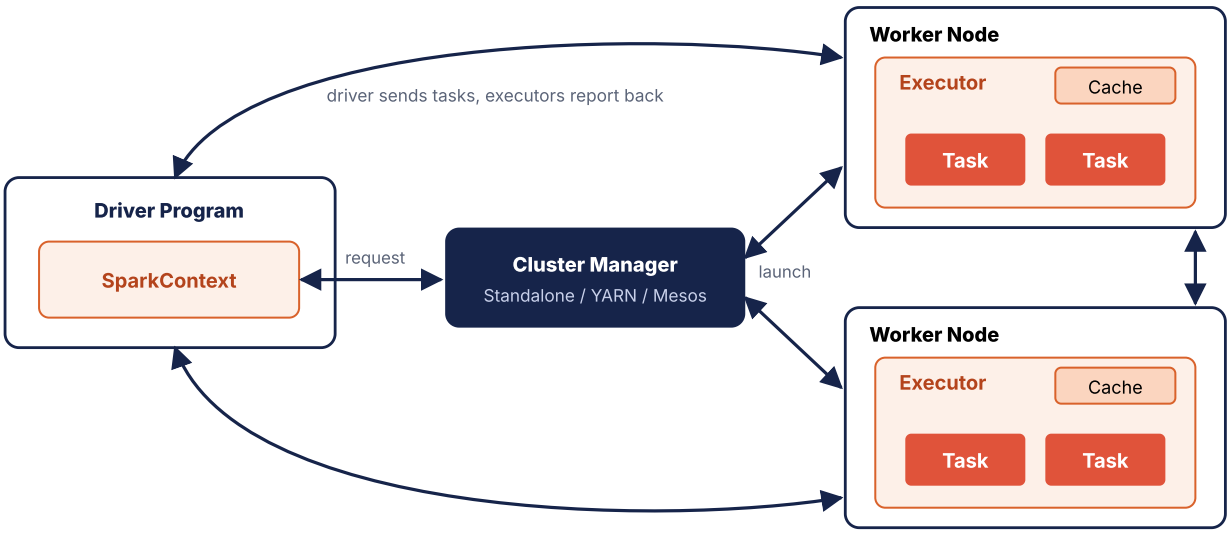


Figure 55 — ✏️ must-draw · Apache Spark architecture

Part	Role
Driver program	Runs the **main()** function and **creates the SparkContext**; does **job scheduling** and **negotiates with the cluster manager**; stores RDD information. Runs in **client or cluster mode**; prepares user code and ships it (as JAR or Python files) to executors.
Cluster manager	**Allocates resources** across applications — **Hadoop YARN, Apache Mesos, Standalone scheduler**. Standalone makes it easy to set up Spark on an empty set of machines.
Worker node	A **slave node** that runs application code.
Executor	A process launched for an application **on a worker node**: **runs tasks**, keeps data in **memory/disk (cache)**, reads and writes external data. Every application has its own executors.
Task	A unit of work sent to one executor.

> ⚠️ **TRAP**
> The **cluster manager only hands out resources** — it does not schedule tasks or run your code. **The driver schedules tasks.**

3.4 How a Spark job executes

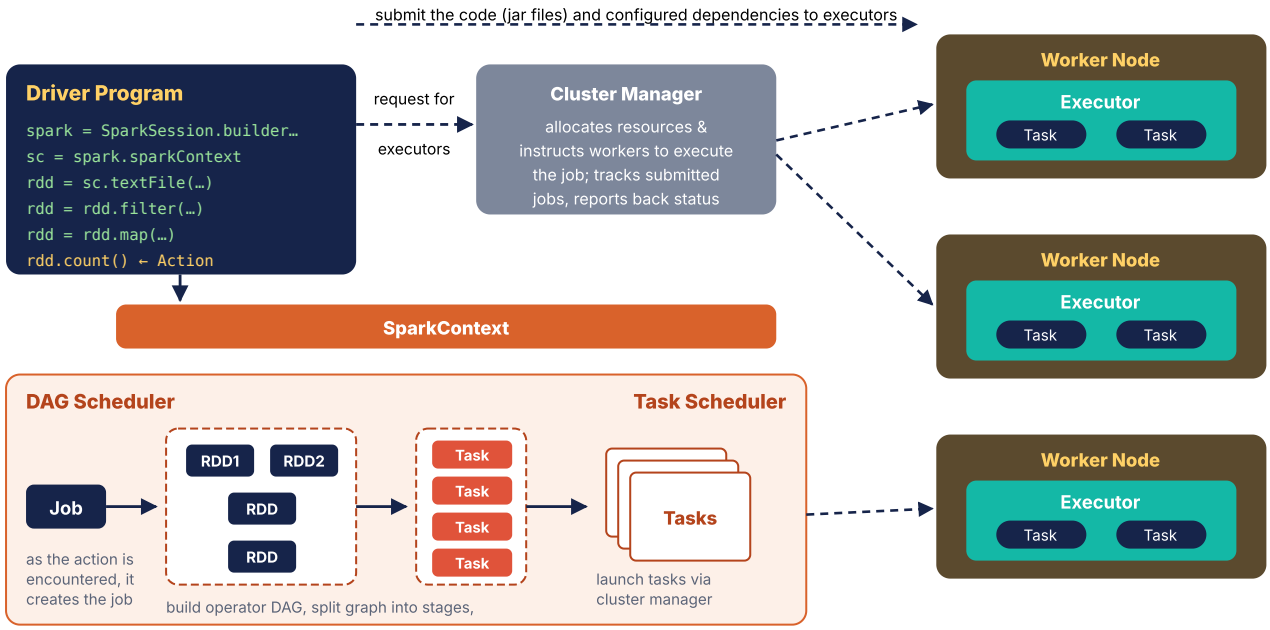


Figure 56 — ✏️ must-draw · Internals of job execution

1. The driver creates a SparkSession/SparkContext and defines operations (read → filter → map) — **nothing runs yet**.
2. When an **action** (e.g., `count()`) appears, a **job** is created.
3. The DAG Scheduler builds the graph of operations (**DAG** = directed acyclic graph, arrows only go forward) and **splits it into stages**.
4. The Task Scheduler **launches the tasks through the cluster manager**.
5. The cluster manager allocates resources and tracks the job; the code (JARs) goes to **executors** on worker nodes, which run the tasks.

Run-time architecture — 10 steps



Figure 57 — Run-time flow of a Spark application

1. Driver **negotiates** resources with the cluster manager.
2. Cluster manager **launches executors** for the driver.
3. Executors **register** with the driver.
4. Code is converted into a **logical DAG**.
5. The DAG is **optimised**.
6. It becomes a **physical execution plan** with stages.
7. Small units called **tasks** are sent to the cluster.
8. Executors **execute** the tasks; the driver **monitors** them.
9. The driver **schedules future tasks** by tracking where cached data is.
10. `stop()` on SparkContext **terminates the executors and releases resources**.

> 🧠 **MEMORY TRICK**
>
> "**N**ew **L**abourers **R**eport; the boss **L**ists, **O**rganises, **P**lans, gives **T**asks; they **E**xecute, he **M**onitors, then **S**ays stop." Same shape as YARN: ask for resources → start workers → run → monitor → release (the driver plays the ApplicationMaster's role).

Spark vs Hadoop MapReduce

	MapReduce	Spark
Processing	Disk-based	**In-memory**
Speed	Slower	Up to 100× faster (memory)
Execution plan	Fixed map → reduce	Optimised **DAG** of stages
Workloads	Batch	Batch, iterative, interactive, streaming
Languages	Mainly Java	Scala, Java, Python, R, SQL
Libraries	Separate (Hive, Pig, Mahout)	Built in (SQL, Streaming, MLlib, GraphX)
Storage	HDFS	**None of its own** — uses HDFS, S3, local files

3.5 Installing and running Spark, cluster managers

Your slides list these topics in the agenda; the details below are from the textbook.

Installing on Ubuntu: (1) install **Java** (JDK) → (2) download the pre-built Spark `.tgz` → (3) extract with `tar -xzf` and move to `/opt/spark` → (4) set `SPARK_HOME` and add `bin` / `sbin` to `PATH` → (5) verify with `pyspark` or `spark-shell --version`. (Python-only shortcut: `pip install pyspark`.) Folders: **bin** (shells, spark-submit, run-

example), **sbin** (start-master/worker), conf, examples, jars, data. Shells: spark-shell (Scala), pyspark, sparkR, spark-sql — `sc` and `spark` are ready-made. Web UI: **localhost:4040**.

First program (shell)

```
>>> lines = sc.textFile("README.md")
>>> lines.count()       # action → job
>>> lines.first()
>>> lines.filter(lambda l: "Spark" in l).count()
```

Word count app

```
counts = (sc.textFile("input.txt")
  .flatMap(lambda l: l.split())
  .map(lambda w: (w, 1))          # like a mapper
  .reduceByKey(lambda a, b: a+b)) # like a
reducer
print(counts.collect()); spark.stop()
$ spark-submit --master local[4] wc.py
```

spark-submit runs an application; main options: `--master` (where to run), `--deploy-mode` (client/cluster), `--class` (Java/Scala main class), executor memory/cores.

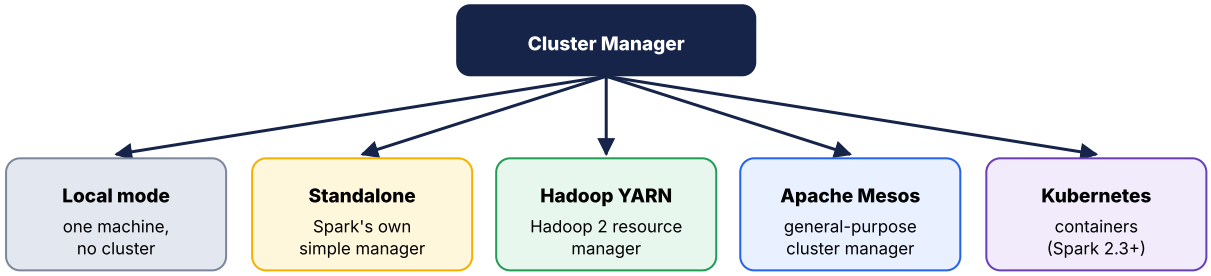


Figure 58 — ✏️ must-draw · Cluster manager types

Type	--master	What it is
Local	local, local[K], local[*]	Everything on one machine (1, K or all-core threads) — learning and testing
Standalone	spark://HOST:7077	**Spark's own** simple manager: one master + workers (UI on 8080)
Hadoop YARN	yarn	Runs Spark on an existing Hadoop cluster, reading HDFS; executors in YARN containers
Apache Mesos	mesos://HOST:5050	General-purpose cluster manager shared with other frameworks
Kubernetes	k8s://https://HOST:PORT	Driver and executors as pods (Spark 2.3+)

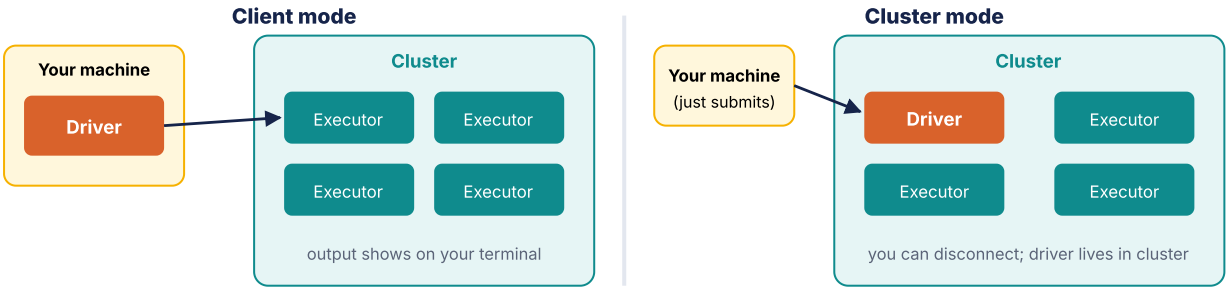


Figure 59 — Client mode vs cluster mode

Client mode: driver runs on the machine that submitted the job (shells always use this; good for interactive work). **Cluster mode:** driver runs inside the cluster (production; you can disconnect). **Running the examples:** `./bin/run-example SparkPi 10` → "Pi is roughly 3.14..." (10 partitions = 10 tasks); Python: `spark-submit examples/src/main/python/pi.py 10`.

🧠 **MEMORY TRICK**

"**L**et **S**park **Y**ell **M**ore **K**indly" — Local, Standalone, YARN, Mesos, Kubernetes. The slides' three: Standalone, YARN, Mesos.

READ LAST

Exam pack

Everything to have fresh in your head as you walk in

How to write answers, the numbers, the likely questions and the classic traps.

How to write a long answer (5–10 marks)

1 · Definition exact textbook words → **2 · Diagram** neat, labelled, numbered arrows → **3 · Points / steps** one heading each, 1–2 lines → **4 · Example** weather data, Amazon, bank → **5 · Conclusion** one line on value

Numbers to remember

Fact	Value	Fact	Value
Unstructured share	80–90%	HDFS block	128 MB
Digital universe 2006 → 2011	0.18 → 1.8 ZB	Replication	3
Disk 1990 vs ~2010	1,370 MB @ 4.4 MB/s (5 min) vs 1 TB @ 100 MB/s (>2.5 h)	NameNode cold start	30+ min
1 TB on 100 disks	< 2 min	Map buffer / spill	100 MB / 80%
Hadoop born	Feb 2006	Reduce copier threads	5
1 TB sort	209 s, 900 nodes	Task attempts	4
MR1 limit	~4,000 nodes	NodeManager timeout	10 min
Spark vs MapReduce	up to 100× (memory)	Reduce slow start	5% maps
NCDC temperature	°C × 10; 9999 = missing	Spark UI / Standalone	4040 / 7077

Most likely questions

Unit 1

1. Classify digital data (+ comparison)
2. Techniques for unstructured data
3. Define Big Data; 3 Vs + other Vs
4. Challenges with / facing Big Data
5. BI vs Big Data
6. What is BDA; Analytics 1.0/2.0/3.0
7. Data scientist skills & roles
8. SMP vs MPP; shared nothing; CAP

Unit 2

1. MapReduce max-temperature flow
2. Design of HDFS; NameNode/DataNode
3. File read (6 steps)
4. File write (7 steps)
5. Federation & HA
6. Classic MR vs YARN job run
7. Failures; Fair vs Capacity
8. Shuffle and sort

Unit 3

1. Spark architecture (diagram)
2. Spark components
3. SparkContext; job/stage/task
4. Job execution / 10-step run-time
5. Cluster managers; client vs cluster
6. Spark vs MapReduce
7. Install & first program

⚠️ CLASSIC TRAPS — CHECK BEFORE YOU HAND IN

- Secondary NameNode is **not** a backup (it checkpoints).
- Cluster manager gives resources; the **driver** schedules tasks.
- Spark has no storage of its own — it reads HDFS etc.
- Velocity ≠ Variability ≠ Volatility.
- "Challenges with Big Data" (6) ≠ "Top challenges facing Big Data" (7).
- Block size = **128 MB**; map output → local disk, reduce output → HDFS.
- XML ↔ SOAP, JSON ↔ REST; UIMA is by IBM.

All the best for CIE-1! 🚀